

Minomaly: A Subgraph Mining Framework for Unsupervised and Interpretable Anomaly Detection in Graphs

Ahmed Youcef Benhalima¹, Seif-Eddine Benkabou^{2*},
Amin Mesmoudi², Allel Hadjali¹

¹ISAE-ENSMA, France.

²Université de Poitiers, France.

Abstract

Detecting structural anomalies in graphs is crucial to uncover unexpected patterns and mitigate risks in fields such as cybersecurity, social network analysis, and biological research. Existing methods, which often rely on machine learning approaches, struggle to balance detection effectiveness, scalability, and interpretability. In this paper, we introduce Minomaly, an unsupervised framework that combines subgraph mining with Graph Neural Networks (GNNs) to detect rare node neighborhoods indicative of structural anomalies. By embedding subgraphs into an Order Embedding Space (OES), Minomaly efficiently estimates subgraph frequencies and employs a greedy search algorithm to expand infrequent node structures, with interpretable and controllable hyperparameters. This approach enables the detection of anomalies in a way that is both interpretable and computationally efficient. Experiments on well-established benchmark datasets demonstrate that Minomaly consistently outperforms [competitive baselines](#) in detection quality and scalability, handling large graphs without memory overhead issues. The implementation of Minomaly along with the associated experimental setup, is available at <https://github.com/gbay7/minomaly>.

Keywords: Graph Anomaly Detection, Unsupervised Learning, Graph Neural Networks, Order Embedding

1 Introduction

Graph Anomaly Detection (GAD) plays a crucial role in uncovering hidden insights and mitigating risks in various domains. For example, in cybersecurity, detecting anomalous network traffic patterns can help identify potential intrusions or malicious activities. In social network analysis, identifying anomalous behaviors can assist in detecting fake accounts, spam, or unusual communication patterns. Similarly, in biological networks, anomaly detection can highlight proteins or genes with unusual interactions, shedding light on disease mechanisms or novel biological processes [1–3]. In financial transaction networks, the presence of cliques might indicate unusual or suspicious activity. For instance, if multiple accounts are found to be tightly connected through transactions (forming a clique), it could suggest fraudulent activity such as money laundering or collusion between accounts (See Figure 1).

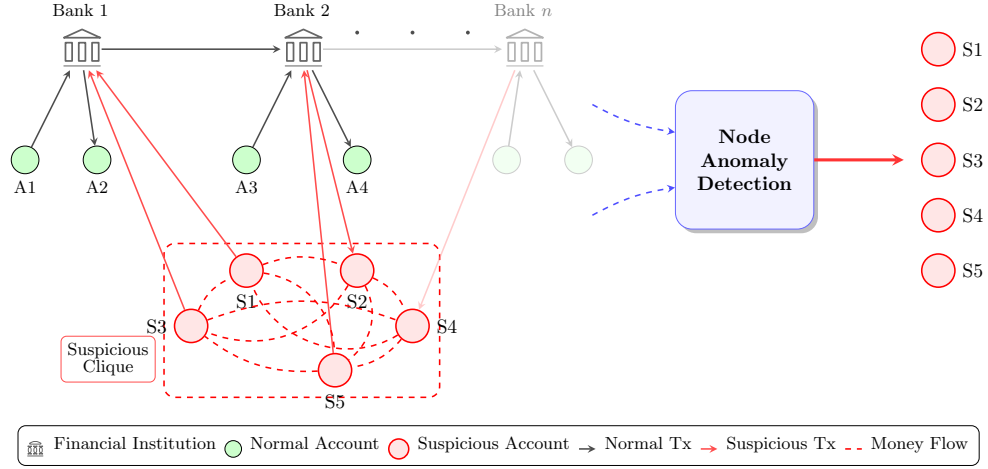


Fig. 1: Financial fraud detection using Graph Anomaly Detection. The figure illustrates how suspicious money flow patterns form cliques among accounts (S1-S5), suggesting potential money laundering activities. A GAD algorithm processes the financial network to identify these unusual structural transaction patterns, enabling financial institutions to flag suspicious accounts for further investigation.

Regardless of the specific application area (e.g., fraud detection, security monitoring), unsupervised node anomaly detection has attracted considerable research attention due to its broad applicability [2, 4]. In graph data, nodes are interconnected through edges to form intricate patterns and structural relationships. These patterns represent interactions, connections, or relationships between entities and often encode meaningful insights relevant to various analytical tasks. The structural arrangement and frequency of these connections can reveal typical behaviors, communities, or clusters, making them critical for identifying deviations indicative of anomalies. Nodes with attribute vectors that deviate considerably from the global attribute norm are

often considered global anomalies. While node attributes provide valuable information, the edges between nodes offer additional insights. If a node belongs to an unusual connection pattern or neighborhood, all nodes within this structure are collectively anomalous and called structural anomalies. When node attributes are considered within this abnormal structure, these anomalies are referred to as contextual anomalies [2, 4].

Our primary focus, however, is on **structural anomalies**. While existing methods have advanced in detecting these anomalies, they often lack generality and interpretability. For instance, BOND [4] considers structural anomalies as densely connected nodes. However, in some real-world geometric graphs, dense connections are common, making this assumption inadequate. Other methods rely on specific motif distributions [5–7], which limits their applicability to diverse real-world contexts such as fraud detection because they do not generalize to all motifs that may exist. Existing unsupervised deep learning techniques often lack solid explanations for their results, limiting their interpretability [8]. Traditionally, frequent pattern outlier detection methods provided interpretable results but were computationally expensive due to the need to enumerate patterns and match similar ones [9–11]. This computational burden arises because the problem reduces to the subgraph isomorphism problem, which is NP-complete. In these methods, frequency serves as a measure of abnormality because rare structures are more likely to be anomalous.

To address these limitations, we propose **Minomaly**, an unsupervised, interpretable, and scalable framework for structural anomaly detection in graphs. Our approach proceeds in two distinct steps:

- **Order Embedding Space (OES) construction** — Minomaly combines subgraph mining with GNN-based order embeddings to encode graph neighborhoods efficiently. This step allows for accurate frequency estimation of subgraphs.
- **Anomalous structures search** — Using the geometrical properties of the OES, Minomaly employs a novel greedy search algorithm to identify rare and unusual node structures, allowing interpretable anomaly detection.

Our contributions can be summarized as follows:

- A new formalization of structural anomaly detection in graphs using subgraph mining. A proof of the correctness of the counting measure used to enumerate anomalous subgraphs, based on the anti-monotonicity property, is provided. Additionally, a generalized definition of structural node anomaly detection is proposed to support anomaly detection across multiple contexts.
- A novel application of order embeddings to graph anomaly detection. Minomaly leverages GNN order embeddings to efficiently encode graph structures.
- **DSAN (Deep Subgraph Anomaly Network)**, an order-embedding model trained on synthetic graphs that transfers to real-world datasets without retraining, with GLASS-style structural labeling and a smooth softplus loss for stable convergence.
- A new optimization technique in anomalous structures search by exploiting the geometrical properties of the OES which significantly reduces the number of nodes that need to be verified during the search for anomalous structures.
- A novel, computationally efficient greedy search algorithm for identifying and interpreting unusual structures in graphs, enabling interpretable GAD.

- An in-depth experimental study on widely used benchmark datasets to validate Minomaly’s performance.

The rest of this paper is organized as follows: Section 2 introduces the related works. Section 3 presents the Minomaly framework in detail while formalizing the problem of node anomaly detection in graphs. Section 4 includes our performance study and experimental results compared with existing approaches. Section 5 highlights key insights from our experiments and discusses their implications. Finally, Section 6 concludes the paper and outlines future research directions.

2 Related Work

Anomaly Detection (AD) research spans diverse data modalities, each presenting unique challenges. Classical methods for tabular data rely on statistical, distance-based, and isolation approaches [3, 12–14], while time series analysis employs techniques designed to capture temporal dependencies [15–18]. Recent deep learning advancements have further refined anomaly detection across these domains [2, 19, 20]. In contrast, graph-structured data introduces additional complexity, as anomalies may emerge within individual nodes, across relationships, or through unexpected structural patterns. Unlike tabular data, where observations are independent, graphs require specialized methods that simultaneously model structural connections and node attributes to identify irregularities effectively.

2.1 Graph Anomaly Detection

Graph Anomaly Detection (GAD) has been extensively studied, employing a range of methods from traditional algorithms to advanced deep learning approaches [2]. These techniques aim to identify anomalous patterns in graph-structured data, which can be critical for applications such as fraud detection, cybersecurity, and biological network analysis. Traditional methods for graph anomaly detection can be divided into two main categories: machine learning-based approaches and pattern mining techniques.

Machine learning methods often rely on matrix decomposition and clustering algorithms to detect anomalies. Radar (residual analysis) [21] identifies anomalies by examining residuals between node attributes and their network coherence, scoring outliers based on reconstruction residual norms. ANOMALOUS (CUR decomposition) [22] applies CUR decomposition [23] and residual analysis to jointly detect anomalies and select relevant node attributes. Clustering-based approaches, such as SCAN (structural clustering) [24], identify irregularities by grouping similar nodes and flagging those that deviate from expected patterns. While these methods provide interpretable results, they often suffer from performance limitations when applied to complex graph structures, particularly in large-scale graphs where scalability remains a challenge. Matrix decomposition operations face high computational complexity in both dense and sparse adjacency matrices, with memory requirements growing quadratically with the number of nodes. Similarly, clustering-based methods require computing pairwise similarities across the entire graph, which becomes prohibitive for graphs with millions of nodes.

Recent advances in deep learning have significantly improved graph anomaly detection [2, 25–27]. In this context, Graph neural networks (GNNs) have demonstrated exceptional performance in various graph mining tasks, making them well-suited for detecting anomalous nodes [4]. These methods leverage representation learning to capture intricate structural dependencies and complex attribute relationships, allowing for more precise node anomaly detection.

Reconstruction-based anomaly detection has gained popularity among deep learning techniques. These models aim to learn the global behavior of node information but often struggle with detecting rare patterns. DOMINANT [28] employs a GCN encoder with separate decoders for structural and attribute reconstruction, while Anomaly-DAE [29] uses two independent autoencoders (AEs) to combine reconstruction losses for anomaly scoring. DONE [5] introduces MLP-based autoencoders for structural and attribute reconstruction, whereas AdONE [5] enhances this method with adversarial learning. Similarly, CONAD [6] leverages graph augmentation and contrastive learning, incorporating human prior knowledge of structural and contextual anomalies. However, reconstruction-based approaches may generate high reconstruction errors for anomalous nodes, as balancing the learning of normal and abnormal patterns remains challenging and poorly controlled, which makes them difficult to interpret due to the black-box nature of deep neural network representations and the lack of direct semantic meaning in reconstruction errors [3].

Generative models offer an alternative to reconstruction-based approaches by leveraging synthetic data to improve anomaly detection. GAAN [30] employs a generator to create synthetic graph nodes, an encoder to map nodes into latent space, and a discriminator to differentiate between real and synthetic nodes. However, these models assume that anomalous nodes resemble synthetic ones, which can reduce their interpretability and generalization capabilities.

A distinct line targets anomalies that take the form of dense subgraphs, which is closely related to our structural setting. Fraudar [31] detects dense fraudulent blocks by greedy peeling on the unattributed graph, with density bounds that are provably robust to camouflage. Spade [32] and its successor [33] incrementalize this idea for real-time fraud detection on evolving graphs at industrial scale. These methods share Minomaly’s attribute-free, structure-only premise, but differ in mechanism: they optimize a global density objective over node blocks, whereas Minomaly measures how rare each node’s local neighborhood is in an order-embedding space and returns the specific rare substructure, which yields node-level interpretability. They are also the natural prior art for the dynamic-graph extension that we discuss as future work.

2.2 Interpretable Anomaly Detection

Interpreting the predictions of graph anomaly detection models is essential for real-world applications. Despite increasing research efforts in explainable artificial intelligence (XAI), most existing work focuses on general GNN explainability methods [34–36]. While these methods provide insights into model behavior and hyperparameter tuning, they are not specifically tailored for anomaly detection. Only a few recent studies [37–42] have addressed the explainability of GAD results, highlighting a significant gap in interpretable anomaly detection techniques.

2.3 Pattern Mining and Order Embedding

Pattern mining offers a complementary, structure-aware view of anomaly detection [7, 43]. Graph pattern mining discovers recurring substructures, from frequent subgraphs to specific motifs. Exact methods such as GSpan [11] and GASTON [44] enumerate all frequent patterns, while approximate approaches like Motivo [9] trade completeness for efficiency. These methods scale exponentially with pattern size, which limits them on large graphs. PANG [43] builds on this line for anomaly detection, converting graphs into vectors of discriminative subgraph patterns mined by GSpan, and GUIDE [7] counts three- and four-node motifs. Both inherit the combinatorial cost of exact mining.

To overcome this cost, recent work learns subgraph relations with GNNs, including FlowSC [45], SPMiner [46], and D2Match [47]. SPMiner is the closest to our work. It embeds subgraphs in an order-embedding space [48, 49] to mine frequent motifs at the graph level. Minomaly reuses this order-embedding idea but repurposes it for a different goal: rather than mining frequent motifs, it estimates the containment frequency of each node’s neighborhood and flags rare ones as anomalies. Minomaly trains its own order-embedding network (DSAN) on synthetic graphs and transfers it to real-world datasets, exploiting the geometrical properties of the OES to grow infrequent neighborhoods into interpretable anomalous structures via a greedy search. We detail this divergence in Section 3.

To address the limitations of existing graph anomaly detection techniques, we introduce Minomaly, a novel framework that redefines anomaly identification by optimally balancing performance, scalability, and interpretability. Unlike conventional methods, Minomaly integrates subgraph mining with GNN-based order embeddings, enabling more precise detection and a deeper understanding of rare structural anomalies in graphs. Specifically, it applies order embeddings to node-level and subgraph-level anomaly detection, leveraging infrequent patterns to identify structural anomalies with interpretable search mechanisms. While **most** existing methods focus solely on providing an anomaly score indicating that a node is structurally anomalous without revealing the specific structure responsible for the anomaly, Minomaly delivers essential interpretable results by explicitly identifying the anomalous structure itself.

3 Minomaly Framework

In this section, we present Minomaly, an approach that uses a GNN for detecting structural anomalies in graphs. Through the GNN, Minomaly embeds graph structures into an Order Embedding Space (OES), where the embeddings provide an approximate frequency count of node structural neighborhoods to identify rare patterns indicative of node anomalies. These detected anomalous structures are identified through a greedy search method, which expands infrequent neighboring graphs using OES, showcasing the interpretability of Minomaly.

For efficiency and effectiveness, a heuristic search is applied to reduce the anomaly search space by selecting potential nodes from the OES that produce anomalous patterns, rather than verifying the normality of all graph nodes.

Overall, the Minomaly framework operates in two steps: **Order Embedding Space Construction** and **Anomalous Structures Search** (See Figure 2).

We formally introduce the foundational graph-theoretic concepts of static graphs (representing the data), induced subgraphs and isomorphism, which serve as the basis for our order-based frequency measure, relying on subgraph matching to count rare node structures. To clarify novelty, Definitions 1–3 restate standard graph-theoretic notions, whereas the notions introduced in this work are the node structure (Definition 4), the order embedding function adapted to subgraph containment, and our frequency measure $Freq_G$ together with its anti-monotonicity property.

Definition 1 (Static Directed Graph). *A static directed graph $G = (V, E)$ consists of a fixed set of nodes V , and edges $E \subseteq V \times V$ which represent the connections between nodes. The order of G is its number of nodes $|V|$, and its size is its number of edges $|E|$.*

Definition 2 (Induced Subgraph). *Let $S \subseteq V$. The induced subgraph $G[S]$ in G by S is the graph $G[S] = (S, E_S)$, where $E_S = \{(u, v) \in E \mid u, v \in S\}$, the set of edges from G that connect two nodes in S . We write $G' \subseteq G$ if G' is subgraph of G , and we denote $\mathcal{G}(G)$ the set of subgraphs of G .*

Definition 3 (Isomorphism). *Let $G[S], G[T] \in \mathcal{G}(G)$. We say $G[S]$ and $G[T]$ are isomorphic, denoted $G[S] \cong G[T]$, if and only if:*

$$\begin{cases} \exists f : S \rightarrow T, f \text{ is bijective,} \\ \forall u_1, u_2 \in S : (u_1, u_2) \in E_S \iff (f(u_1), f(u_2)) \in E_T. \end{cases}$$

To provide a general definition of structural anomalies, beyond those based on specific assumptions [4–7], we propose our concept of a node structure:

Definition 4 (Node Structure). *Let $u \in V$. The **structure** or **neighborhood graph** of the node u is the connected subgraph induced by the set of nodes that are reachable from u , including u itself.*

- The term *reachable* refers to the nodes that can be reached by traversing edges starting from u .
- This subgraph has a limited size, determined by the size of the neighborhood within a defined distance or number of nodes.
- Ego graphs refer to one-hop neighbors, which is more restrictive.

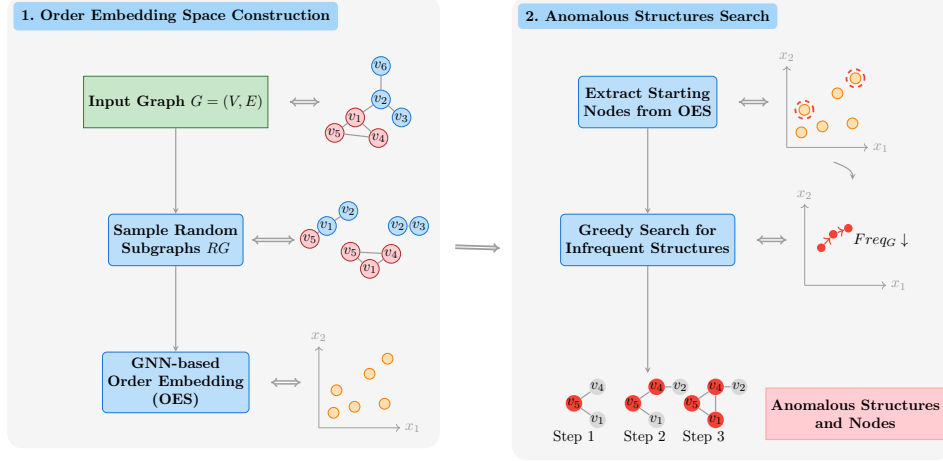


Fig. 2: Minomaly Framework

3.1 Order Embedding

Here, we introduce our method for estimating the **frequency** of small subgraphs to identify anomalous node structures based on their rarity. Under the assumption that rare structures are more likely to be anomalous [1, 3], we use subgraph containment frequency as our measure of abnormality. We restrict attention to small subgraphs because anomalous signals are local, visible within a node's immediate neighborhood, and small subgraphs keep frequency estimation tractable, avoiding the combinatorial blow-up of large-pattern mining.

Intuitively, the frequency of a subgraph refers to the number of subgraphs that match it isomorphically. Finding these matches reduces to the NP-complete subgraph isomorphism problem [50]. To address this, we first tackle the **subgraph matching** problem by training a GNN to determine whether a given graph is a subgraph of another.

The subgraph matching problem naturally exhibits a partial order structure: if $G_1 \subseteq G_2$, then any graph containing G_2 must also contain G_1 . Order embeddings provide an ideal framework for capturing this hierarchical containment relationship, allowing us to efficiently approximate the NP-complete subgraph isomorphism problem through learned continuous representations. By preserving the partial order in the embedding space, order embeddings enable efficient frequency estimation via geometric operations, transforming the discrete combinatorial problem into a continuous optimization task. Given these partial order properties of subgraph matching, we use **order embeddings** to map these graphs and capture these properties. We formalize this encoding through our proposed order embedding function:

Definition 5 (Order Embedding Function). An *order embedding* function for graphs, denoted as $\phi : \mathcal{G}(G) \rightarrow \mathbb{R}^d$, is an injective function that maps a subgraph of G

275 into a d -dimensional embedding space such that for two subgraphs $G_1, G_2 \in \mathcal{G}(G)$:

$$G_1 \subsetneq G_2 \iff \phi(G_1) \preceq \phi(G_2), \quad \text{such that } G_1 \subsetneq G_2 \text{ means } \exists G_{iso} \subseteq G_2 : G_{iso} \cong G_1$$

276 Here, the partial order $\preceq$ on the embedding space is defined component-wise, mean-
 277 ing $\phi(G_1) \preceq \phi(G_2)$ if and only if $\phi_i(G_1) \leq \phi_i(G_2), \forall i \in \llbracket 1, d \rrbracket$ such that $\phi =$
 278 $(\phi_1, \phi_2, \dots, \phi_d)$. The partial order $\subsetneq$ on $\mathcal{G}(G)$ denotes subgraph isomorphism.

279 3.1.1 Subgraph Matching

280 In contrast to existing deep learning methods, Minomaly trains its own order-
 281 embedding model, which we call **DSAN** (Deep Subgraph Anomaly Network), for
 282 subgraph matching once on synthetic graphs and transfers it across all target datasets
 283 without per-dataset retraining. While SPMiner [46] uses order embeddings to mine
 284 frequent motifs at the graph level, Minomaly repurposes this idea for the opposite
 285 goal: estimating the containment frequency of each node’s neighborhood and flagging
 286 rare ones as anomalies. DSAN is trained from scratch on graphs sampled from a mix
 287 of random graph models, namely Erdős-Rényi graphs [51], Watts-Strogatz graphs [52],
 288 Barabási-Albert graphs [53], and Power Law Cluster graphs [54]. This synthetic mix
 289 exposes the model to diverse local topologies, so the learned order embedding gen-
 290 eralizes to real-world graphs. Two design choices make DSAN an accurate subgraph
 291 matcher: a GLASS-style node labeling that sharpens its structural expressiveness, and
 292 a smooth order-embedding loss that stabilizes training. We describe both below.

293 As illustrated in Figure 3, the trained model employs GNN with a multi-layer per-
 294 ceptron (MLP) for binary classification ($G_1 \subseteq G_2$). It takes two graphs, G_1 and G_2 ,
 295 as inputs. For the GNN architecture, we employ GraphSAGE [55] due to its sampling-
 296 based neighborhood aggregation, which enables scalable processing of large graphs by
 297 avoiding the full neighborhood expansion required by other GNN architectures. The
 298 model processes the graph structure through 8 layers of GraphSAGE-based graph
 299 convolutions with learnable skip connections [55]. To make the matcher sensitive to
 300 which substructure is being queried, DSAN adopts the labeling trick of GLASS [56]:
 301 every node carries a two-dimensional structural label, an anchor indicator and an
 302 inside-subgraph indicator. The first marks the node from which the neighborhood is
 303 anchored, and the second marks the nodes that belong to the queried subgraph. This
 304 labeling supplies boundary information that plain node features cannot convey, and it
 305 provably increases the expressive power of message-passing GNNs on subgraph-level
 306 tasks [56], sharpening the containment relation that the order embedding must cap-
 307 ture. A post-processing layer with linear transformations and Leaky ReLU activations
 308 produces the embeddings $\phi(G_1)$ and $\phi(G_2)$. These embeddings are concatenated and
 309 passed through an MLP to generate the output. The goal is to train the network to
 310 learn an order embedding function ϕ that respects its definition. To train the model,
 311 a set of positive and negative training instances are generated. For positive examples,
 312 we sample random subgraphs G_1^+ and G_2^+ from the graph G such that $G_1^+ \subseteq G_2^+$. For
 313 negative examples, we add enough random nodes or edges to a subgraph G_1^- to gener-
 314 ate G_2^- , or we can use any other valid random generation. The base order-embedding

penalty [49] measures the violation of the partial order in the embedding space:

$$E(G_1, G_2) = \sum_{i \in [1, d]} \|\max(0, \phi_i(G_1) - \phi_i(G_2))\|^2$$

For positive pairs $G_1^+ \subseteq G_2^+$ this violation is minimized directly, driving $\phi(G_1^+) \preceq \phi(G_2^+)$. For negative pairs, rather than the hinge penalty $\max(0, \alpha - E)$ used in prior order-embedding work, DSAN minimizes a smooth *softplus* penalty. The total loss is

$$\sum_{(G_1^+, G_2^+) \in \text{Positives}} E(G_1^+, G_2^+) + \sum_{(G_1^-, G_2^-) \in \text{Negatives}} \text{softplus}(\alpha - E(G_1^-, G_2^-)), \quad (1)$$

where $\text{softplus}(x) = \log(1 + e^x)$ and $\alpha > 0$ is the margin that separates positive and negative penalties. The advantage of the softplus formulation is that it remains differentiable everywhere. The hinge penalty has exactly zero gradient once a negative pair exceeds the margin, so easy negatives stop contributing to learning; the softplus keeps a small but non-vanishing gradient beyond the margin. This continuous signal yields sharper separation between subgraph and non-subgraph pairs and more stable convergence, which in turn improves the accuracy of the containment-frequency estimates that drive the anomaly search.

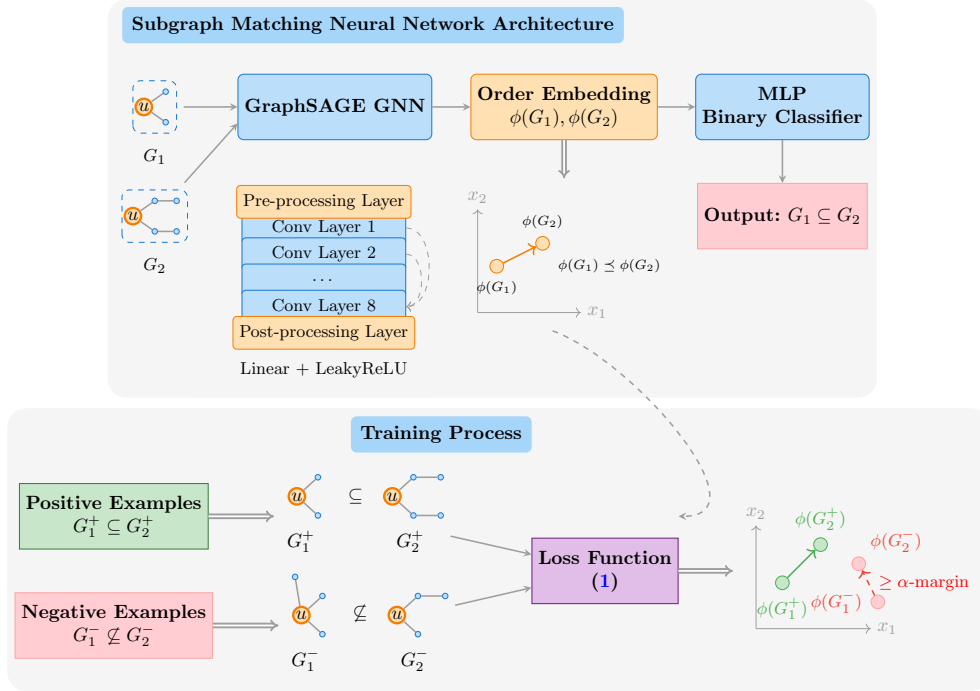


Fig. 3: Subgraph Matching Neural Network Architecture

Definition 6 (Anchor Node). *For implementation purposes in the neural network architecture, we define the **anchor node** as a special node u from which the subgraph is defined.*

In subgraph matching, the anchor node plays a crucial role in guiding the GNN to focus on a localized neighborhood, ensuring consistency in subgraph extraction and embedding. By centering the subgraph around an anchor node, the GNN learns structural patterns relative to a fixed reference point, facilitating more robust and meaningful comparisons in the OES.

3.1.2 Frequency Count

The primary function of the subgraph matching model based on Order Embedding is to determine whether a given graph is a subgraph of another, enabling the quantification of its frequency within the entire graph. This is represented by any function $freq_G : \mathcal{G}(G) \rightarrow \mathbb{N}$, which measures the existence of its similar (isomorphic) subgraphs in G .

Building upon the order embedding framework, we now introduce our frequency measure $Freq_G$, which is based on matching subgraphs in the graph. It is defined as the number of graphs that contain a matching of G' ($|UP_G(G')|$). Our proposed measure is formalized as follows:

Definition 7 (Proposed Frequency Measure). *For any $G' \in \mathcal{G}(G)$, we define:*

$$\begin{cases} UP_G(G') &= \{G_{up} \subseteq G \mid \exists G_{iso} \subseteq G_{up} : G_{iso} \cong G'\} \\ &= \{G_{up} \subseteq G \mid \phi(G') \preceq \phi(G_{up})\}, \\ Freq_G(G') &= |UP_G(G')|. \end{cases}$$

The correctness of frequency measures in pattern mining is traditionally evaluated through the anti-monotonicity property [57], a well-established criterion that ensures consistency in frequency-based anomaly detection.

Definition 8 (Anti-monotonicity Counting Property). *$freq_G$ is anti-monotonic, if for all $G_1, G_2 \in \mathcal{G}(G)$:*

$$G_1 \subseteq G_2 \implies freq_G(G_2) \leq freq_G(G_1)$$

This implies that the frequency of a graph is never greater than the frequency of any of its subgraphs. Consequently, if a graph is infrequent, all graphs containing it are also infrequent.

Proposition 1. *The proposed frequency measure $Freq_G$ is anti-monotonic.*

Proof. Let $G_1, G_2 \in \mathcal{G}(G)$ such that $G_1 = (S_1, E_1)$, $G_2 = (S_2, E_2)$ and $G_1 \subseteq G_2$.

- If $G_{up} \in UP_G(G_2)$, then $\exists G_{iso} \subseteq G_{up}$ such that $G_{iso} \cong G_2$.
- Since $G_1 \subseteq G_2$, $\exists G'_{iso} \subseteq G_{iso} \subseteq G_{up}$ such that $G'_{iso} \cong G_1$ (by restricting the bijection $f : S_2 \rightarrow T$ from Definition 3 to $S_1 \subseteq S_2$).
- Thus, $G_{up} \in UP_G(G_1)$, and we have $UP_G(G_2) \subseteq UP_G(G_1)$.

360 Taking cardinalities: $|UP_G(G_2)| \leq |UP_G(G_1)|$, i.e. $Freq_G(G_2) \leq Freq_G(G_1)$, there-
 361 fore, $Freq_G$ is anti-monotonic. $\square$

362 To estimate this frequency $Freq_G$ using the trained GNN model —since computing
 363 the exact $Freq_G$ via subgraph matching is NP-complete—, the approach leverages
 364 an efficient approximation. As illustrated in Figure 4, the method first decomposes
 365 the graph G into random subgraphs $RG = \{G'_1, G'_2, \dots, G'_k\}$ of different sizes from
 366 `min_neigh_size` to `max_neigh_size`, k must be sufficiently large. Subgraph sampling
 367 follows the tree sampling approach [58], which extracts connected neighborhoods by
 368 randomly selecting an anchor node and performing breadth-first traversal until the
 369 desired size is reached. These subgraphs are then processed through the GNN model to
 370 generate their respective order embeddings in order to construct the Order Embedding
 371 Space (OES) that is used for counting subgraph occurrences. The approach of subgraph
 372 matching is considered advantageous for these reasons:

- 373 • **Generalizability** — The model shows the generalization test performance of
 374 subgraph matching across real-world datasets, by training only on a general bal-
 375 anced synthetic dataset. On the other hand, the model only handles subgraph
 376 matching without considering features.
- 377 • **OES geometric properties** — Using the partial order properties of the
 378 embeddings, the frequency of a subgraph G' can be quantified by counting the
 379 upper-right embeddings of RG graphs. Additionally, the anti-monotonicity prop-
 380 erty can be verified by how the space is organized according to graph size and
 381 order.

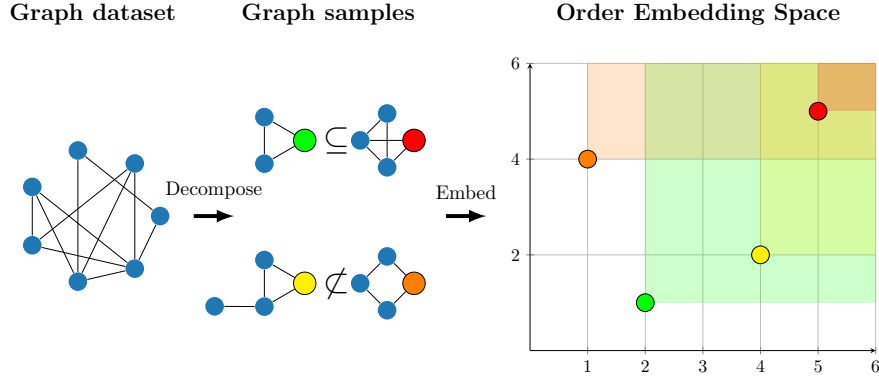


Fig. 4: Order Embedding Space construction process

382 After constructing the OES, the frequency of a subgraph G' , $Freq_G(G')$, is
 383 determined by counting the number of the random embedded subgraph G'_i from
 384 $RG = \{G'_1, G'_2, \dots, G'_k\}$ in the space, where $\phi(G') \preceq \phi(G'_i)$ as predicted by the model's
 385 output layer.

$$Freq_G(G') = \frac{|\{G'_i \in RG \mid \phi(G') \preceq \phi(G'_i)\}|}{k}$$

386 The embeddings $\phi(G'_i)$ correspond to the upper-right points of $\phi(G')$ in the geometric
 387 space OES (see Figure 5).

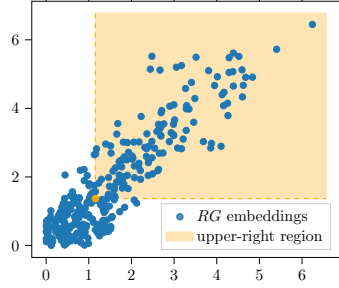


Fig. 5: Geometric visualization of the count prediction procedure in the OES

388 To experimentally verify the anti-monotonicity of the estimated $Freq_G$, we ana-
 389 lyze the results shown in Figure 6. The trained GNN model, applied to the Cora
 390 dataset (see Table 2), organizes the space based on subgraph size (number of edges)
 391 and order (number of nodes), as larger graphs cannot be subgraphs of smaller ones.
 392 This observation indicates that $Freq_G$ exhibits anti-monotonicity, supporting its
 393 correctness.

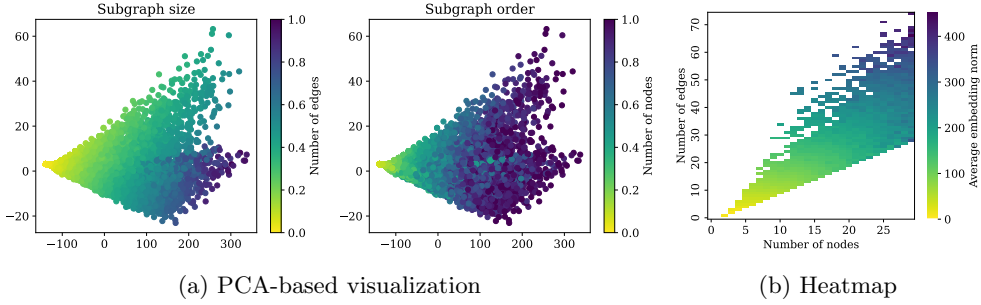


Fig. 6: (a) PCA visualization of order embeddings for random neighborhoods of 1–30 nodes from the Cora dataset. PCA preserves straight lines, indicating subgraph containment of a point lies within a quadrilateral region [46]. (b) The heatmap shows average embedding norms by graph size and order, highlighting structure-embedding relationships. Lighter shades are for smaller graphs, darker for larger ones by size or order.

394 3.2 Anomalous Structures

395 Defining and estimating the frequency of subgraphs, along with verifying its correct-
 396 ness through the anti-monotonicity property on node-anchored structures, provides a

foundational step in identifying structural node anomalies based on the rarity of their neighborhoods. To achieve this, we formally propose our general definition of structural anomalies using subgraph frequency, without imposing any specific subgraph patterns as in [46, 59]:

Definition 9 (Structural Anomaly). *Let $u \in V$. The node u is a **structural anomaly** of G if and only if u belongs to an unexpected structure graph of u .*

- *We define an unexpected graph $G' = (V', E')$ in G as a rare subgraph in terms of graph frequency $\text{freq}_G : \mathcal{G}(G) \rightarrow \mathbb{N}$ that quantifies the existence of its similar (isomorphic) subgraphs in G :*

$$\text{freq}_G(G') < T_F \quad \text{with} \quad T_F > 0$$

- *All nodes $u' \in V'$ of the graph G' are structural anomalies.*

Given the estimated Freq_G from the order embedding function ϕ and the OES, the goal of the search procedure is to identify rare node-anchored structures within the given graph. As illustrated in Figure 7, Minomaly detects anomalous small neighborhoods by iteratively expanding infrequent structures through the addition of neighboring nodes. Subgraphs that meet a certain frequency threshold T_F are identified as structural anomalies. The resulting graph represents the anomalies, demonstrating the interpretability of the method.

The approach adopts a greedy search (See Algorithm 1 and Figure 7d) for anomalous structures G' , beginning from a starting node u and growing by iteratively selecting a candidate node u' from the current neighborhood frontier to obtain the most infrequent structure at each step. To avoid scoring each candidate against all k reference embeddings, the search draws a fixed random sample of $m \ll k$ references once and estimates frequency from this sample throughout. This process continues until one of the following conditions is met: $\text{Freq}_G(G') < T_F$, the maximum subgraph order (`max_steps`) is reached, or no significant change is observed (`max_no_change`). If the first condition is satisfied, the final small structure and all its nodes are identified as anomalies based on the final frequency, with the anomaly score defined as $1 - \text{Freq}_G(G')$.

Algorithm 1 Greedy Search for Anomalous Structures

Require: Graph G , starting node u , threshold T_F , max_steps , max_no_change , max_cands , sample size m , OES references $\mathcal{R} = \{\phi(G'_1), \dots, \phi(G'_k)\}$

Ensure: Anomalous structure G' , Anomaly score score

- 1: Draw a uniform random sample $\mathcal{S} \subset \mathcal{R}$, $|\mathcal{S}| = m$
- 2: Initialize $G' \leftarrow \{u\}$, $\text{steps} \leftarrow 0$, $\text{no_change_steps} \leftarrow 0$, $\text{score} \leftarrow 0$
- 3: **while** $\text{steps} < \text{max_steps}$ **do**
- 4: $\text{steps} \leftarrow \text{steps} + 1$
- 5: Initialize $\text{best_freq} \leftarrow \infty$, $G'' \leftarrow G'$
- 6: Sample a subset of max_cands candidate nodes from the neighborhood frontier of G'
- 7: Embed all candidates in a single batch: $\{\phi(G' \cup \{v\})\}_{v \in \text{cands}}$
- 8: **for** each sampled node v **do**
- 9: $G_{\text{temp}} \leftarrow G' \cup \{v\}$
- 10: $\widehat{\text{Freq}}_G(G_{\text{temp}}) \leftarrow \frac{|\{e \in \mathcal{S} | \phi(G_{\text{temp}}) \preceq e\}|}{m}$
- 11: **if** $\widehat{\text{Freq}}_G(G_{\text{temp}}) < \text{best_freq}$ **then**
- 12: $\text{best_freq} \leftarrow \widehat{\text{Freq}}_G(G_{\text{temp}})$
- 13: $G'' \leftarrow G_{\text{temp}}$
- 14: $G' \leftarrow G''$
- 15: **if** $\widehat{\text{Freq}}_G(G') < T_F$ **then**
- 16: Mark G' and all its nodes as anomalies
- 17: **break**
- 18: **if** $\widehat{\text{Freq}}_G(G')$ unchanged **then**
- 19: $\text{no_change_steps} \leftarrow \text{no_change_steps} + 1$
- 20: **else**
- 21: $\text{no_change_steps} \leftarrow 0$
- 22: **if** $\text{no_change_steps} \geq \text{max_no_change}$ **then**
- 23: **break**
- 24: $\text{score} \leftarrow 1 - \widehat{\text{Freq}}_G(G')$ **return** G' , score

425 **Note on Candidate Sampling:** In Algorithm 1 (line 6), candidate nodes are sampled using
 426 Weighted Random Sampling. Weights are based on node visit frequency during neighborhood
 427 exploration, with frequently visited nodes receiving higher weights as they contribute to
 428 infrequent patterns. Other relevant sampling strategies could also be applied.

429 **Note on Reference Sampling:** In Algorithm 1 (line 1), a fixed random sample of
 430 m reference embeddings is drawn once from the k embedded neighborhoods. All frequency
 431 estimates within the search use this same sample, reducing the per-candidate scoring cost from
 432 $O(kd)$ to $O(md)$. Because the sample is drawn uniformly, $\widehat{\text{Freq}}_G$ is an unbiased estimator
 433 of Freq_G with standard error $O(1/\sqrt{m})$.

434 This search can be viewed as a monotonic walk in the search space of OES,
 435 which also demonstrates the preserved anti-monotonicity property due to the orga-
 436 nized structure of the OES (See Figure 7c). If the resulting neighborhood G' of u
 437 is grown by the ordered sequence of subgraphs $G_u^{(1)} \subseteq G_u^{(2)} \subseteq \dots \subseteq G_u^{(l)}$ such that
 438 $l \in \llbracket 1, \text{max_steps} \rrbracket$ and $G' = G_u^{(l)}$, a monotonic path of embedding search $\phi(G_u^{(1)}) \preceq$
 439 $\phi(G_u^{(2)}) \preceq \dots \preceq \phi(G_u^{(l)})$ is observed, which in turn shows the anti-monotonicity of
 440 Freq_G :

$$\text{Freq}_G(G_u^{(l)}) \leq \dots \leq \text{Freq}_G(G_u^{(2)}) \leq \text{Freq}_G(G_u^{(1)})$$

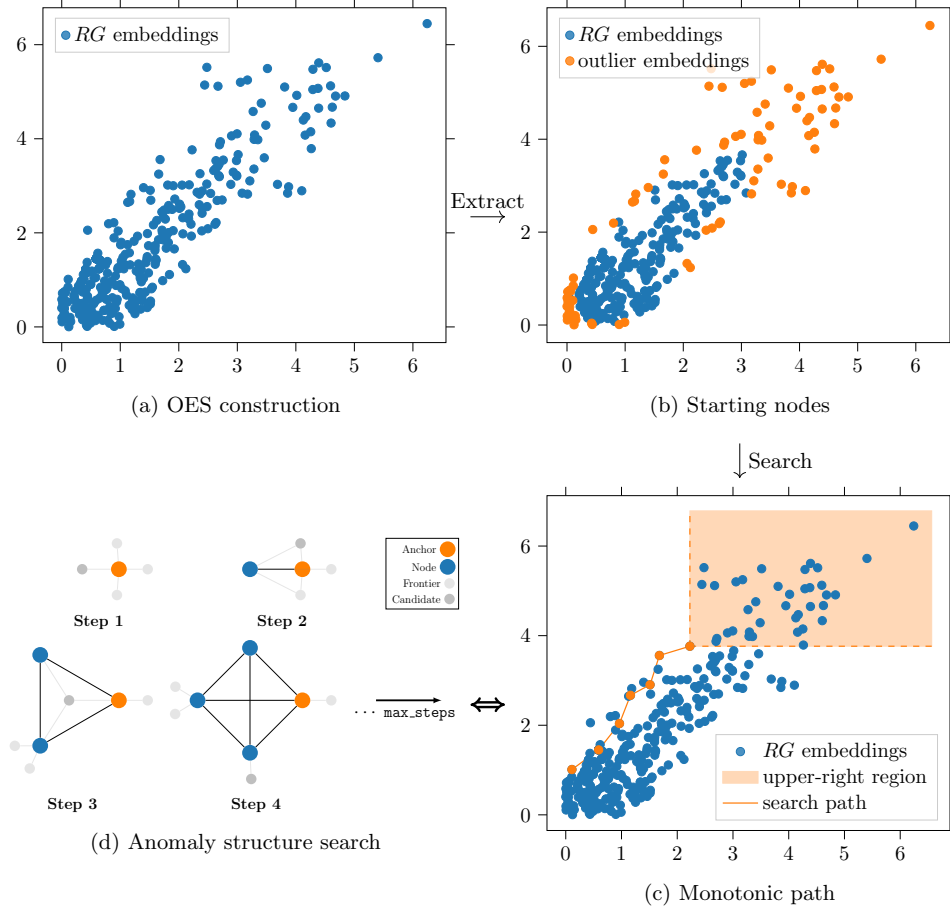


Fig. 7: OES construction and anomalous structure search procedures

- 441 The Algorithm 1 is not applied to all graph nodes, but to potential **starting nodes**
 442 (See Figure 7b) that are extracted from the OES in two ways:
 443 (a) **OES infrequent embedded subgraphs** — The infrequent neighborhoods G'_i
 444 from $RG = \{G'_1, G'_2, \dots, G'_k\}$ can be checked using the count prediction func-
 445 tion $Freq_G(G'_i) < T_F$. The resulting anchors of these subgraphs are considered
 446 anomalous. Two types of neighborhoods can be selected: small infrequent ones,
 447 and larger ones that are infrequent due to the anti-monotonicity property, which
 448 may also contain smaller subgraphs contributing to their infrequency.
 449 (b) **OES frontier** — Taking advantage of the geometric properties of OES, isolated
 450 and boundary points are more likely not to have upper-right points, which can be
 451 approximately detected by an outlier detection method like Isolation Forest [60].
 452 These points are isolated due to the margin hyperparameter α , which separates
 453 them from the rest of the embeddings.

In addition, this method can be used to find anomalous structures of **node anomalies detected by other methods** that lack interpretations or have low precision by considering these anomalies as starting nodes.

4 Experiments

In this section, we evaluate Minomally through extensive experiments. We begin by outlining the experimental protocol, which includes a description of the datasets used, an introduction to the representative node anomaly detection methods for comparison, and a discussion of the evaluation metrics commonly employed in anomaly detection. Next, we present the results, comparing the performance of our approach with the baseline methods. Finally, the performance study examines the impact of dataset characteristics and hyperparameters, along with an evaluation of interpretability, scalability, and the efficiency of our search method.

4.1 Experimental Protocol

This section describes the experimental setup used to evaluate Minomally. Our experiments were conducted on a Linux machine with Python 3.12.2, CUDA 12, and an NVIDIA RTX A2000 GPU (12 GB). We train our own order-embedding model (DSAN) for subgraph matching, following the order-embedding learning procedure used for pattern mining [46, 49]. Minomally was implemented using PyTorch 2.4 and Torch Geometric 2.6, while the benchmarked methods were implemented following [61]. We performed our benchmark following the protocol outlined in [4], using the hyperparameter grid and datasets provided in the paper.

We set the Minomally hyperparameters shown in Table 1 as configurations (C_1 , A_1 , F_1) for each dataset, fixing the OES dimension to $d = 64$, `min_neigh_size` to 1, and `max_neigh_size` to 30. Additionally, we utilized 8 threads for the search phase. `batch_size` for Minomally refers to the number of embedded RG subgraphs processed in a single batch.

These hyperparameters, are interpretable and manually tuned to improve results. This allows refinement after each run by filtering user-relative positives, expanding search spaces, or reducing computational resources, without random hyperparameter search.

Minomally’s order-embedding model (DSAN), trained from scratch on synthetic graphs with margin $\alpha = 0.1$, achieves 95% accuracy in determining whether one graph is a subgraph of another. A single inference query costs $O(Ld^2s_{\max})$ where L is the number of GNN layers, d the embedding dimension, and $s_{\max}$ the maximum neighborhood size, all fixed hyperparameters independent of the input graph size. The same trained model is reused across all benchmarked datasets without retraining.

Table 1: Minomally Hyperparameters Configurations

Parameter	C_1	A_1	F_1
Dataset	Cora	Amazon	Flickr
k	10,000	50,000	150,000
m	10,000	10,000	10,000
T_F	$\frac{35}{2708}$ ≈ 0.013	$\frac{150}{13752}$ ≈ 0.011	$\frac{1000}{89250}$ ≈ 0.011
max_steps	7	11	5
max_no_change	5	3	3
max_cands	All	20	10
batch_size	1,000	10,000	10,000

4.1.1 Datasets

To evaluate Minomally, we use the following datasets, which are provided by [4, 61] and have been used in previous works [7, 28–30, 62]:

- **Cora** [63] : The Cora dataset is a network of machine learning papers, represented as a directed graph that illustrates the citation relationships between papers.
- **Amazon** [64] : The Amazon Computers network consists of nodes representing products, with edges indicating that two products are frequently purchased together.
- **Flickr** [65] : The Flickr dataset is a network of online images from Flickr, with links formed between images that share common properties.

Table 2: Dataset Statistics

Dataset	Nodes	Edges	Avg. Degree	Anomalies
Cora	2,708	11,060	4.1	70 (5.1%)
Amazon	13,752	515,042	37.2	350 (5.0%)
Flickr	89,250	933,804	10.5	2,240 (4.9%)

The table 2 summarizes the key characteristics of the three datasets used in our experiments. It presents the number of nodes and edges for each dataset, the average node degree (which reflects the typical connectivity), and the number (and percentage) of anomalies detected.

These datasets offer diverse graph structures—from the sparse connectivity of Cora to the high interconnectivity of Amazon, with Flickr balancing large size and moderate connectivity. The consistent anomaly ratio (around 5%) across all datasets facilitates a fair evaluation of the performance and robustness of various anomaly detection methods.

4.1.2 Compared Methods

The following [established baselines](#) were selected to benchmark Minomally’s performance, representing diverse deep learning approaches to GAD. For the Cora and Amazon datasets, we employed full-batch training with 400 epochs, while the

larger Flickr dataset required mini-batch processing with batch size 64 and reduced training to just 1 epoch with 3-hop neighborhood sampling. Hidden dimensions of $\{32, 64, 128, 256\}$ were explored for all three datasets. Across all methods, we maintained consistent regularization with dropout rates $\{0, 0.1, 0.3\}$, learning rates $\{0.01, 0.05, 0.1\}$, and weight decay of 0.01. We tuned the balance weight $\alpha \in \{0.2, 0.5, 0.8\}$ between reconstruction and structural losses, following the protocol of [4]. Hyperparameters were randomly selected over 20 trials, and we report the performance as the mean and standard deviation across these trials.

- **GCNAE** [4, 66]: An autoencoder using GCNs as encoder and decoder to learn node embeddings from graph structure and attributes. Anomaly scores are based on reconstruction error.
- **DOMINANT** [28]: Uses a GCN encoder and dual decoders - one for node attribute reconstruction and another for graph structure reconstruction. The final anomaly score combines both reconstruction errors to capture different aspects of abnormality.
- **AnomalyDAE** [29]: Similar to DOMINANT but employs two separate autoencoders (AEs). Its structural encoder uniquely processes both adjacency information and node features simultaneously, while its attribute decoder leverages both structural and attribute embeddings for reconstruction.
- **DONE** [5]: Uses separate MLP-based autoencoders for structure and attributes, with a unified optimization objective that simultaneously learns node embeddings and anomaly scores through a joint loss function.
- **AdONE** [5]: Extends DONE by incorporating adversarial learning with a discriminator that works to align structural and attribute embeddings in the latent space.
- **CONAD** [6]: Leverages graph augmentation and contrastive learning based on human prior knowledge of structural and contextual anomalies. Anomalies are scored using a combination of contrastive loss and graph reconstruction error.

4.1.3 Evaluation Metrics

To comprehensively assess anomaly detection performance, we employed standard binary classification metrics that are particularly relevant for imbalanced datasets typical in anomaly detection. In the context of anomaly detection, true positives (TP) represent correctly identified anomalies, false positives (FP) are normal nodes incorrectly flagged as anomalies, true negatives (TN) are correctly identified normal nodes, and false negatives (FN) are anomalies that were missed. These metrics evaluate both the ranking quality of anomaly scores and the classification performance at various thresholds:

- **Precision**: It measures the proportion of true positives among the predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** (or True Positive Rate TPR) It measures the proportion of true positives among the actual positives.

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN}$$

- **F1 Score:** It is the harmonic mean of precision and recall. Balances recall and precision, providing a single measure of overall performance.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **False Positive Rate (FPR):** It measures the proportion of false positives among the actual negatives.

$$\text{FPR} = \frac{FP}{FP + TN}$$

- **Average Precision (AP):** It summarizes the precision-recall curve as the weighted mean of precision achieved at each threshold, with the increase in recall from the previous threshold used as the weight.

$$\text{AP} = \sum_n (R_n - R_{n-1}) \times P_n$$

where P_n and R_n are the precision and recall at the n th threshold.

- **Area Under the ROC Curve (AUC):** It measures the entire two-dimensional area underneath the ROC curve, which plots TPR against FPR at different classification thresholds. The AUC ranges from 0 to 1, where 0.5 represents random classification performance, and 1.0 represents perfect classification.

4.2 Results

Table 3 presents a comprehensive comparison between Minomally and competitive GAD baselines across three datasets of varying size and complexity. Minomally demonstrates superior performance across all evaluation metrics, achieving the best average rank (1.0) in every metric: AUC, F1, AP, Recall, and Precision.

On the Cora dataset, Minomally achieves outstanding F1, AP, Recall, and Precision scores, significantly outperforming all competitors, with impressive margins of 60.8 percentage points in F1 and 72.1 percentage points in AP compared to the second-best method. For the Amazon dataset, Minomally demonstrates even greater dominance, surpassing all alternatives across every metric, with particularly notable improvements in F1 (68.5% versus 21.7% for the next best) and AP (57.3% versus 12.2%). Perhaps most significantly, Minomally is the only method capable of effectively handling the large-scale Flickr dataset, where most competing approaches fail due to memory limitations. Only GCNAE could execute on Flickr but performed poorly (49.9% AUC versus Minomally’s 91.0%).

Table 3: Comparison of AUC, F1, AP, Recall, and Precision across methods

Dataset	Model	AUC	F1	AP	Recall	Precision
Cora	Minomaly	97.1 $\pm$ 0.0 (Rank: 1.0)	97.1 $\pm$ 0.0 (Rank: 1.0)	94.4 $\pm$ 0.0 (Rank: 1.0)	94.3 $\pm$ 0.0 (Rank: 1.0)	100.0 $\pm$ 0.0 (Rank: 1.0)
	GCNAE	52.5 $\pm$ 0.0 (Rank: 7.0)	5.3 $\pm$ 0.0 (Rank: 7.0)	3.1 $\pm$ 0.0 (Rank: 7.0)	12.9 $\pm$ 0.0 (Rank: 7.0)	3.3 $\pm$ 0.0 (Rank: 7.0)
	DOMINANT	92.1 $\pm$ 9.3 (Rank: 3.0)	32.3 $\pm$ 10.6 (Rank: 3.0)	19.9 $\pm$ 6.5 (Rank: 4.0)	78.7 $\pm$ 25.9 (Rank: 3.0)	20.3 $\pm$ 6.7 (Rank: 3.0)
	DONE	90.1 $\pm$ 5.9 (Rank: 5.0)	20.2 $\pm$ 11.0 (Rank: 5.0)	22.7 $\pm$ 18.8 (Rank: 2.0)	49.2 $\pm$ 26.9 (Rank: 5.0)	12.7 $\pm$ 6.9 (Rank: 5.0)
	AdONE	91.1 $\pm$ 5.0 (Rank: 4.0)	25.7 $\pm$ 11.1 (Rank: 4.0)	16.3 $\pm$ 7.1 (Rank: 5.0)	62.6 $\pm$ 27.0 (Rank: 4.0)	16.2 $\pm$ 7.0 (Rank: 4.0)
	AnomalyDAE	81.7 $\pm$ 11.8 (Rank: 6.0)	16.7 $\pm$ 14.1 (Rank: 6.0)	11.7 $\pm$ 8.3 (Rank: 6.0)	40.6 $\pm$ 34.3 (Rank: 6.0)	10.5 $\pm$ 8.9 (Rank: 6.0)
	CONAD	95.4 $\pm$ 1.2 (Rank: 1.0)	36.3 $\pm$ 3.9 (Rank: 2.0)	22.3 $\pm$ 2.8 (Rank: 3.0)	88.5 $\pm$ 9.5 (Rank: 2.0)	22.9 $\pm$ 2.4 (Rank: 2.0)
Amazon	Minomaly	97.6 $\pm$ 0.0 (Rank: 1.0)	68.5 $\pm$ 0.0 (Rank: 1.0)	57.3 $\pm$ 0.0 (Rank: 1.0)	97.1 $\pm$ 0.0 (Rank: 1.0)	52.9 $\pm$ 0.0 (Rank: 1.0)
	GCNAE	49.0 $\pm$ 0.0 (Rank: 7.0)	3.7 $\pm$ 0.0 (Rank: 7.0)	2.5 $\pm$ 0.0 (Rank: 7.0)	9.1 $\pm$ 0.0 (Rank: 7.0)	2.3 $\pm$ 0.0 (Rank: 7.0)
	DOMINANT	91.4 $\pm$ 0.1 (Rank: 2.0)	21.6 $\pm$ 0.2 (Rank: 3.0)	12.2 $\pm$ 0.1 (Rank: 2.0)	53.1 $\pm$ 0.4 (Rank: 3.0)	13.5 $\pm$ 0.1 (Rank: 3.0)
	DONE	87.3 $\pm$ 4.0 (Rank: 5.0)	17.8 $\pm$ 6.8 (Rank: 5.0)	11.7 $\pm$ 4.4 (Rank: 5.0)	43.9 $\pm$ 16.8 (Rank: 5.0)	11.2 $\pm$ 4.3 (Rank: 5.0)
	AdONE	85.5 $\pm$ 5.6 (Rank: 6.0)	13.2 $\pm$ 7.2 (Rank: 6.0)	9.3 $\pm$ 4.0 (Rank: 6.0)	32.6 $\pm$ 17.8 (Rank: 6.0)	8.3 $\pm$ 4.5 (Rank: 6.0)
	AnomalyDAE	89.7 $\pm$ 5.8 (Rank: 4.0)	21.7 $\pm$ 3.4 (Rank: 2.0)	12.0 $\pm$ 2.0 (Rank: 4.0)	53.4 $\pm$ 8.4 (Rank: 2.0)	13.6 $\pm$ 2.1 (Rank: 2.0)
	CONAD	91.3 $\pm$ 0.1 (Rank: 3.0)	21.5 $\pm$ 0.2 (Rank: 4.0)	12.2 $\pm$ 0.1 (Rank: 3.0)	53.1 $\pm$ 0.4 (Rank: 4.0)	13.5 $\pm$ 0.1 (Rank: 4.0)
Flickr	Minomaly	91.0 $\pm$ 0.0 (Rank: 1.0)	87.3 $\pm$ 0.0 (Rank: 1.0)	77.7 $\pm$ 0.0 (Rank: 1.0)	82.1 $\pm$ 0.0 (Rank: 1.0)	93.3 $\pm$ 0.0 (Rank: 1.0)
	GCNAE	49.9 $\pm$ 0.1 (Rank: 2.0)	4.4 $\pm$ 0.1 (Rank: 2.0)	2.6 $\pm$ 0.0 (Rank: 2.0)	10.9 $\pm$ 0.2 (Rank: 2.0)	2.7 $\pm$ 0.0 (Rank: 2.0)
	Others	Out of GPU Memory				
Avg. Rank	Minomaly	1.0	1.0	1.0	1.0	1.0
	GCNAE	5.3	5.3	5.3	5.3	5.3
	DOMINANT	2.5	3.0	3.0	3.0	3.0
	DONE	5.0	5.0	3.5	5.0	5.0
	AdONE	5.0	5.0	5.5	5.0	5.0
	AnomalyDAE	4.0	4.0	5.0	4.0	4.0
	CONAD	2.0	3.0	3.0	3.0	3.0

4.2.1 Effect of the DSAN matcher

To evaluate the impact of our DSAN matcher, we compare it against the SPMiner order-embedding model [46] integrated into the same Minomaly pipeline, with the search procedure held fixed. Both models share the same SkipLastGNN architecture (8 layers, 64 dimensions, GraphSAGE convolutions) but differ in key design choices, summarized in Table 4. DSAN uses GLASS-style two-dimensional structural labeling (anchor indicator and subgraph membership) instead of the single anchor bit, a smooth softplus order-embedding loss instead of the hinge penalty, and a more diverse training set covering five random graph generators instead of three. Despite requiring $10\times$ fewer training batches (100,000 vs. 1,000,000), DSAN achieves stronger detection, as shown in Table 5. DSAN improves detection on Cora (AUC $95.3 \rightarrow 97.1$, AP $74.6 \rightarrow 94.4$) and Amazon (AUC $95.4 \rightarrow 97.6$, Recall $92.9 \rightarrow 97.1$), and matches SPMiner on Flickr.

Table 4: Training configurations of the SPMiner matcher and our DSAN model.

	SPMiner	DSAN (ours)
Encoder	SkipLastGNN (fixed skip)	SkipLastGNN (GLASS)
Conv type	SAGE	SAGE
Layers / dim	8 / 64	8 / 64
Input dim	1 (anchor bit only)	2 (anchor + subgraph)
Loss function	Hinge: $\max(0, \alpha - E)$	Softplus: $\log(1 + e^{\alpha - E})$
Training batches	1,000,000	100,000
Generators	3 (ER, WS, BA)	5 (+ PLC, dense ER)
Graph sizes	6–30	6–30

Table 5: Comparison of the SPMiner matcher and our DSAN model integrated into the same Minomaly pipeline, with the search procedure fixed.

Dataset	Matcher	AUC	F1	AP	Recall	Precision
Cora	SPMiner (original)	95.3	87.1	74.6	91.4	83.1
	DSAN (ours)	97.1	97.1	94.4	94.3	100.0
Amazon	SPMiner (original)	95.4	69.6	57.7	92.9	55.7
	DSAN (ours)	97.6	68.5	57.3	97.1	52.9
Flickr	SPMiner (original)	90.5	87.3	77.7	81.2	94.3
	DSAN (ours)	91.0	87.3	77.7	82.1	93.3

From the Precision-Recall curves in Figure 8, we observe low Precision@k across all methods (precisions in left side region), which corresponds to low-frequency scores detected as anomalies for Minomaly. This is attributed to the existence of some unlabeled structures that are rarer than the labeled ones, as shown in the interpretations (See Figure 9b). DONE achieves high Precision@k on Cora due to its restrictive anomaly definition but fails to maintain precision on other datasets and for higher recalls, unlike our frequency-based method. This demonstrates that using frequency as a measure of abnormality is more effective in identifying true anomalies, surpassing reconstruction-loss methods that struggle with rare patterns and fail to capture anomalous patterns across different contexts, as mentioned in Section 2.

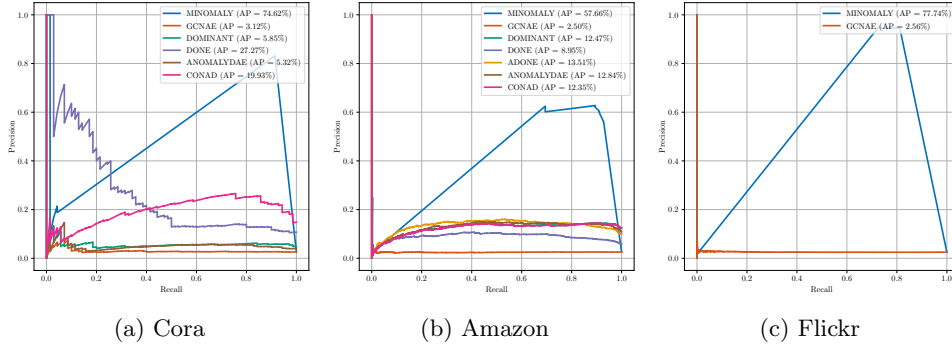


Fig. 8: Precision-Recall curves for different models.

4.3 Performance Study

Complexity analysis.

We analyze the two phases of Minomaly. Let k be the number of sampled reference neighborhoods, d the embedding dimension, L the number of GNN layers, $s_{\max}$ the maximum neighborhood size, m the reference-sample size used for scoring, and $|S|$ the number of starting nodes. Since L , d , and $s_{\max}$ are fixed hyperparameters, a single GNN forward pass on one bounded neighborhood costs

$$c_\phi = O(L s_{\max}^2 d^2),$$

a constant that does not depend on the graph size $|V|$ or $|E|$.

OES construction (performed once). Drawing the k reference neighborhoods by tree sampling costs $O(k s_{\max}^2)$, and embedding them costs $O(k c_\phi)$, which dominates. Selecting the $|S|$ starting nodes scores the k embeddings with the outlier detectors: the order-embedding detector compares each embedding against the reference sample, $O(k m d)$, and the isolation-forest detector adds $O(k \log k \cdot d)$. This phase is computed once and reused for every subsequent query.

Frequency estimation. Estimating $Freq_G$ for a pattern reduces to one geometric comparison of its embedding against the m references, $O(m d)$, which replaces the NP-complete subgraph-matching test; with a fixed sample size m this is constant per query.

Anomalous-structure search. Starting from $|S|$ nodes, the greedy search runs $T = \text{max_steps}$ steps; each step expands n_b beams by up to $b = \text{max_cands}$ candidates, embedding each candidate (c_ϕ) and scoring it against the references ($O(m d)$). The total search cost is

$$O(|S| \cdot T \cdot n_b \cdot b \cdot (c_\phi + m d)).$$

Space. Minomaly stores the graph as a sparse CSR adjacency, $O(|V| + |E|)$, the k reference embeddings, $O(k d)$, and the GNN parameters, $O(L d^2)$; the pairwise violation tensors are processed in fixed-size chunks. No $|V| \times |V|$ matrix is ever materialized.

Two consequences support our scalability claim. First, because neighborhoods are bounded by $s_{\max}$, every per-neighborhood operation (sampling, embedding, comparison) is constant in $|V|$ and $|E|$; the cost grows only with the controllable counts k and $|S|$, and the heuristic starting-node selection keeps $|S| \ll |V|$ (Table 7 shows between 49% and 80% of nodes are processed). Second, the memory footprint is linear in the graph size, $O(|V| + |E| + k d)$, in contrast with reconstruction-based detectors whose adjacency reconstruction grows as $O(|V|^2)$; this is why Minomaly scales to Flickr, where those baselines exhaust memory.

In this section, we provide a deep comprehensive analysis of Minomaly’s performance characteristics beyond comparative benchmarking. First, we evaluate the effectiveness of our starting node selection strategy, quantifying its computational benefits and impact on detection accuracy by examining how the size k of RG affects performance on Flickr. Next, we analyze how the `max_cands` parameter balances effectiveness and efficiency across different levels of graph connectivity. Additionally, we

present interpretable visualizations of detected anomalous structures and their frequency evolution. Finally, we conduct a detailed hyperparameter study, examining how TF and `max_steps` influence detection performance across various evaluation metrics, revealing clear trends that facilitate practical parameter tuning.

To analyze the impact of the size k of RG on detection quality and its significance in covering patterns within large datasets, we extend Table 1 by introducing two additional configurations, F_2 and F_3 , for the Flickr dataset, as shown in Table 6.

Table 6: Minomaly Flickr Hyperparameters

Parameter	Run F_1	Run F_2	Run F_3
Dataset	Flickr	Flickr	Flickr
k	150,000	200,000	250,000
m	10,000	10,000	10,000
TF	$\frac{1000}{89250}$ ≈ 0.011	$\frac{1000}{89250}$ ≈ 0.011	$\frac{1000}{89250}$ ≈ 0.011
<code>max_steps</code>	5	5	5
<code>max_no_change</code>	3	3	3
<code>max_cands</code>	10	10	10
<code>batch_size</code>	10,000	12,500	12,500

The results corresponding to each hyperparameter configuration (C_1 , A_1 , F_1 , F_2 , F_3) are detailed in Table 7.

Table 7: Minomaly Results

Result	Run C_1	Run A_1	Run F_1	Run F_2	Run F_3
Dataset	Cora	Amazon	Flickr	Flickr	Flickr
Starting nodes	$\frac{2152}{2708}$ $\approx 79.5\%$	$\frac{9935}{13752}$ $\approx 72.2\%$	$\frac{43582}{89250}$ $\approx 48.8\%$	$\frac{47648}{89250}$ $\approx 53.4\%$	$\frac{54614}{89250}$ $\approx 61.2\%$
Positive starting nodes	$\frac{69}{70}$ $\approx 98.6\%$	$\frac{312}{350}$ $\approx 89.1\%$	$\frac{1733}{2240}$ $\approx 77.4\%$	$\frac{1833}{2240}$ $\approx 81.8\%$	$\frac{1977}{2240}$ $\approx 88.3\%$
AUC	97.14%	97.62%	90.98%	91.85%	93.17%
AP	94.43%	57.30%	77.71%	79.77%	82.92%
Recall	94.29%	97.14%	82.10%	83.84%	86.47%
Precision	100.00%	52.88%	93.26%	93.90%	93.53%
Search time (s)	142	610	306	310	353
OES build time (s)	22	118	542	1021	1335

Impact of Starting Nodes Strategy and RG Size

Table 7 demonstrates the quantitative impact of our starting nodes selection strategy on both computational efficiency and detection performance. By selecting a subset of nodes as starting points based on methods (a) *OES infrequent embedded sub-graphs* and (b) *OES frontier* detection, described in 3.2—rather than examining all

nodes—Minomally reduces computational complexity while maintaining high detection quality. For the larger Flickr dataset, the algorithm processes between 48.8% and 61.2% of nodes depending on the reference set size k .

The precision of these selected starting nodes is quantifiably high, with 98.6% of true anomalies in Cora and 89.1% in Amazon recovered among the starting nodes. In Flickr, as parameter k increases from 175,000 in Run F₁ to 250,000 in Run F₃, we observe a corresponding increase in the proportion of positive starting nodes from 77.4% to 88.3%. This improved node selection directly correlates with enhanced detection performance: AUC increases from 90.98% to 93.17%, AP from 77.71% to 82.92%, and recall from 82.10% to 86.47% across the three Flickr runs.

This performance improvement introduces a computational trade-off: as the proportion of starting nodes in Flickr increases, search time increases from 306s to 353s. The consistently high precision (93.26%–93.90% across all Flickr runs) confirms that the approach minimizes unnecessary searches in regions prone to false positives, thus optimizing the balance between computational cost and detection effectiveness. These results validate that the strategic selection of starting nodes from anomalous neighborhoods effectively reduces the search space while preserving the algorithm’s node anomaly detection capabilities.

Impact of max_cands Parameter

Table 7 and Table 2 together reveal important insights about the `max_cands` hyperparameter’s effect on performance and computational efficiency across datasets with varying connectivity characteristics. This parameter, as shown in Algorithm 1, limits the number of candidate nodes considered from the neighborhood frontier during the greedy search process.

For the Cora dataset, with its relatively low average degree (4.1), all frontier nodes were considered during the search (`max_cands` = "All"). This complete frontier exploration was computationally feasible due to Cora’s limited connectivity and starting nodes, resulting in the shortest search time (293s) among the three datasets despite examining all possible candidates.

In contrast, for the Amazon dataset with its high connectivity (average degree 37.2), restricting `max_cands` to 20 was necessary to maintain computational tractability. Despite this significant frontier sampling restriction, the model achieved an AUC of 97.62% and a recall of 97.14%, demonstrating that examining only a smaller subset of the densely connected frontier can yield strong detection performance.

For Flickr, the largest dataset with moderate connectivity (average degree 10.5), `max_cands` was further restricted to 10. The model achieved AUC scores between 90.98% and 93.17% across different runs. The search time increased with the number of starting nodes examined (from 306s to 353s), but remained manageable despite the dataset’s scale.

The experimental findings suggest that even restrictive frontier sampling (examining only 10-20 candidates) can achieve strong anomaly detection performance on large, dense graphs, as the search still identifies structurally meaningful anomalous patterns despite not examining all possible expansions at each step. This is due to the `max_neigh_size` size of the random subgraphs that was limited to 30, so the OES was

702 meaningful enough to extract the properties of the diversity of small neighborhoods,
703 that enabled also parallel frequency calculations and significantly reduces the number
704 of candidate nodes (`max_cands`) to grow anomalous node structures.

705 *Extracted Anomalous Patterns*

706 Tables 8, 9, and 10 present the predicted anomalous pattern groups for the three
707 datasets. After identifying anomalous structures in each dataset, we applied Weisfeiler-
708 Lehman (WL) graph hashing to group structurally isomorphic patterns together; all
709 nodes of these discovered structures are predicted as anomalous. Each table shows
710 representative graphs for these pattern groups, where **red nodes** indicate true anomalous
711 nodes and **green nodes** indicate true normal nodes from the ground-truth. The
712 **Count** field shows how many times each pattern appears in the dataset, while **A/N**
713 denotes the composition of true anomalous versus true normal nodes within each
714 representative pattern.

715 The pattern extraction results demonstrate Minomally’s strong interpretability
716 advantage, enabling users to understand and validate detected anomalies through
717 visual structural analysis. For Cora (Table 8), the dominant pattern 7-0 (Count: 31,
718 A/N: 7/0) reveals a fully connected clique structure consisting entirely of anomalous
719 nodes, accounting for the majority of detected anomalies and contributing to the
720 method’s high precision (100.00%) on this dataset. Flickr (Table 10) exhibits similar
721 behavior with patterns 7-0, 7-1, and 7-2 (Count: 375, 49, and 6 respectively) showing
722 pure anomalous structures, supporting the excellent precision (93.26%–93.90%)
723 observed in Table 7. In contrast, Amazon (Table 9) displays more complex pattern
724 diversity with 10 distinct groups, where patterns like 11-1, 11-2, and 11-4 through
725 11-9 contain only normal nodes (A/N: 0/11) despite being structurally complex and
726 similar to anomalous patterns, explaining the lower precision (52.88%) while maintaining
727 excellent recall (97.14%). This visual pattern interpretation capability allows
728 domain experts to manually filter false positives by examining structural characteristics
729 and node compositions, effectively transforming the anomaly detection task into an
730 interpretable pattern recognition problem. The visualization enables users to quickly
731 identify which patterns merit further investigation versus those that can be safely filtered,
732 significantly enhancing the practical utility of the detection results beyond raw
733 performance metrics.

Table 8: Extracted anomalous patterns for Cora dataset

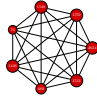
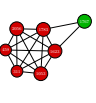
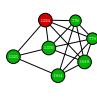
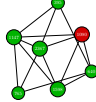
7-0		7-1		7-2		7-3		7-4	
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats
	Count: 31 A/N: 7/0		Count: 3 A/N: 0/7		Count: 1 A/N: 1/6		Count: 1 A/N: 6/1		Count: 1 A/N: 0/7
7-5		7-6		7-7		7-8			
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats		
	Count: 1 A/N: 1/6		Count: 1 A/N: 1/6		Count: 1 A/N: 1/6		Count: 1 A/N: 0/7		
 True anomalous node  True normal node									

Table 9: Extracted anomalous patterns for Amazon dataset

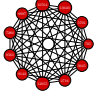
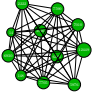
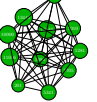
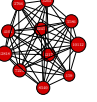
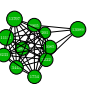
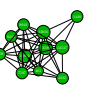
11-0		11-1		11-2		11-3		11-4	
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats
	Count: 96 A/N: 11/0		Count: 20 A/N: 0/11		Count: 10 A/N: 0/11		Count: 10 A/N: 11/0		Count: 6 A/N: 0/11
11-5		11-6		11-7		11-8		11-9	
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats
	Count: 5 A/N: 0/11		Count: 4 A/N: 0/11		Count: 3 A/N: 0/11		Count: 2 A/N: 0/11		Count: 2 A/N: 0/11
● True anomalous node ● True normal node									

Table 10: Extracted anomalous patterns for Flickr dataset

7-0		7-1		7-2		7-3		7-4	
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats
	Count: 375 A/N: 7/0		Count: 49 A/N: 0/7		Count: 6 A/N: 7/0		Count: 3 A/N: 6/1		Count: 2 A/N: 0/7
7-5		7-6		7-7		7-8		7-9	
Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats	Pattern	Stats
	Count: 2 A/N: 0/7		Count: 1 A/N: 1/6		Count: 1 A/N: 0/7		Count: 1 A/N: 0/7		Count: 1 A/N: 1/6
● True anomalous node ● True normal node									

Interpretations

While Minomaly demonstrates strong overall performance across datasets (Table 7), Amazon’s precision (52.88%) is notably lower than that of Cora (100.00%) and Flickr (93.26%–93.90%). This discrepancy can be attributed to two primary factors revealed through our pattern interpretation analysis. Figure 9a, 9b and 9c illustrate examples of these interpretations. First, Amazon contains numerous structural patterns that are labeled as normal in the ground truth but exhibit strong structural similarities to true anomalous patterns. Second, nodes connected to anomalous structures are labeled as normal in the ground truth and can be labeled as anomalies in certain contexts. Despite these precision challenges, the method maintains excellent recall (97.14%) on Amazon, demonstrating its effectiveness at identifying the complete set of true anomalies.

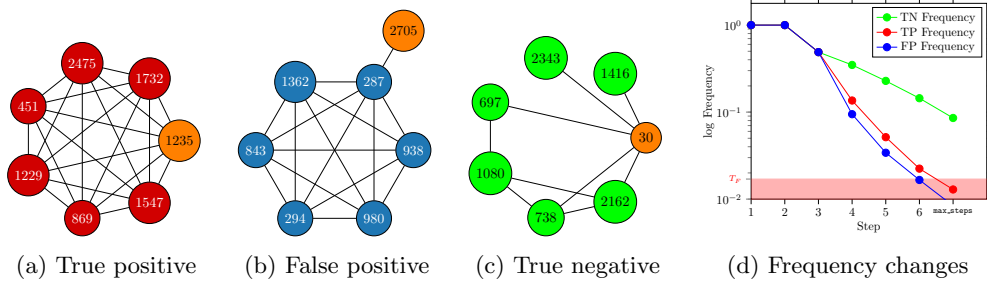


Fig. 9: Minomaly interpretation results on the Cora dataset, illustrating three examples: an anomalous structure (red), a normal one (green), and a falsely predicted node connected to an anomalous structure (blue), along with their frequency change curves.

Figure 10 visually demonstrates the effectiveness of the method in identifying anomalous regions by step-walking over the embedded anomalous neighborhoods,

747 which contributes to achieving high recall (92.86%) on the all datasets. Addition-
748 ally, the overall high precision observed across datasets indicates that the approach
749 effectively filters out true positive nodes from the large visualized searched region.

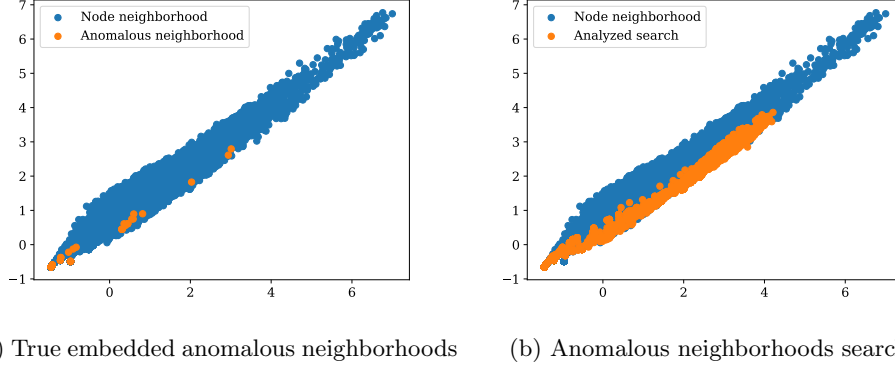


Fig. 10: This figure shows the true anomalous neighborhood embeddings and their corresponding search analysis by the algorithm in the OES of Cora, visualized in two dimensions.

750 T_F and `max_steps` Hyperparameters Study

751 We study the effect of T_F and `max_steps` on AUC, AP, Recall, and Precision met-
752 rics, evaluating the anomaly detection framework on the Cora and Amazon, Flickr
753 datasets (Figures 11, 12 and 13). The ranges of hyperparameters for each dataset are
754 summarized in Table 11.

Table 11: Hyperparameter ranges for different datasets

Dataset	Cora	Amazon	Flickr
Total nodes	2,708	13,752	89,250
T_F range (unnormalized)	$\llbracket 1, 40 \rrbracket$	$\{10, 20, \dots, 200\}$	$\{100, 200, \dots, 1400\}$
<code>max_steps</code> range	$\llbracket 1, 11 \rrbracket$	$\llbracket 1, 21 \rrbracket$	$\llbracket 1, 11 \rrbracket$

755 T_F and `max_steps` control the threshold used to filter anomalous nodes from the
756 search results, as shown in Figure 9d. Thus, it is important to study their effect on
757 detection performance.

758 The heatmaps indicate that Minomaly is highly sensitive to these hyperparameters,
759 and performance can be improved only by tuning them, as they serve as key thresholds
760 for the framework (See Figures 11, 12, 13 and 9d).

761 Since AUC, AP, Recall, and F1 metrics share similar heatmaps, it is sufficient to
762 use only AUC for manual tuning, as it is a reliable metric for this task. In contrast,
763 the baseline methods that rely on reconstruction thresholds typically yield suboptimal
764 results for the other metrics, despite having good AUC (See Table 3), because their
765 reconstruction thresholds are independent of the hyperparameters, and choosing the
766 right reconstruction threshold is another task to consider.

767 We also observe that precision is generally high across the entire search grid and
 768 shows little sensitivity to the hyperparameters, as illustrated in the heatmaps 11c, 12c
 769 and 13c, but it decreases for higher values of T_F and max_steps . This aligns with our
 770 focus on detecting rare small node neighborhoods.

771 The AUC surfaces for both datasets are relatively smooth, with a few local maxima,
 772 as shown in Figures 11f, 12f and 13f. The AUC heatmap shows optimal hyperparam-
 773 eters of $T_F = \frac{24}{2708} \approx 0.88\%$ and $\text{max_steps} = 9$ for Cora, $T_F = \frac{150}{13752} \approx 1.1\%$ and
 774 $\text{max_steps} = 11$ for Amazon, and $T_F = \frac{1000}{89250} \approx 1.1\%$ and $\text{max_steps} = 5$ for Flickr.
 775 These values are close to the used benchmark parameters in Table 1 and are within
 776 the region of interest. The AUC heatmap for Cora (Figure 11a), Amazon (Figure 12a)
 777 and Flickr (13a) show that high scores are achieved in a relatively continuous linear
 778 region of T_F and max_steps . Thus, finding the optimal values is straightforward. For
 779 small continuous values of T_F and max_steps , performance is high and stable, but it
 decreases outside these regions.

781 Finding these regions is not difficult: the frequency threshold T_F is typically small
 782 since anomalies are rare and can be determined from the dataset characteristics.
 783 max_steps is also small, as we focus on small neighborhoods, followed by manual tun-
 784 ing based on the interpretations. Alternatively, a faster automatic solution without
 785 varying the parameters in the greedy search is to predefine the search region by setting
 786 sufficiently high values for max_steps and T_F , then execute Minomally with these val-
 787 ues. The frequency curves will be generated and filtered for each frequency $\text{freq} < T_F$
 788 and step $\text{step} < \text{max_steps}$. The choice of optimal hyperparameters from freq and
 789 step is then left to the user based on the interpretations.

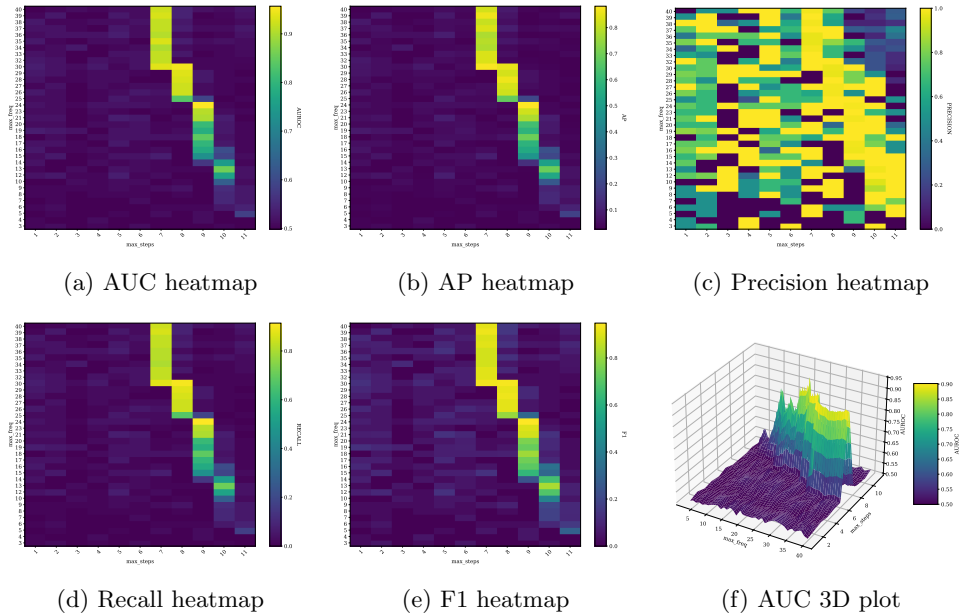


Fig. 11: T_F (y-axis) and max_steps (x-axis) parameter analysis on the Cora dataset.

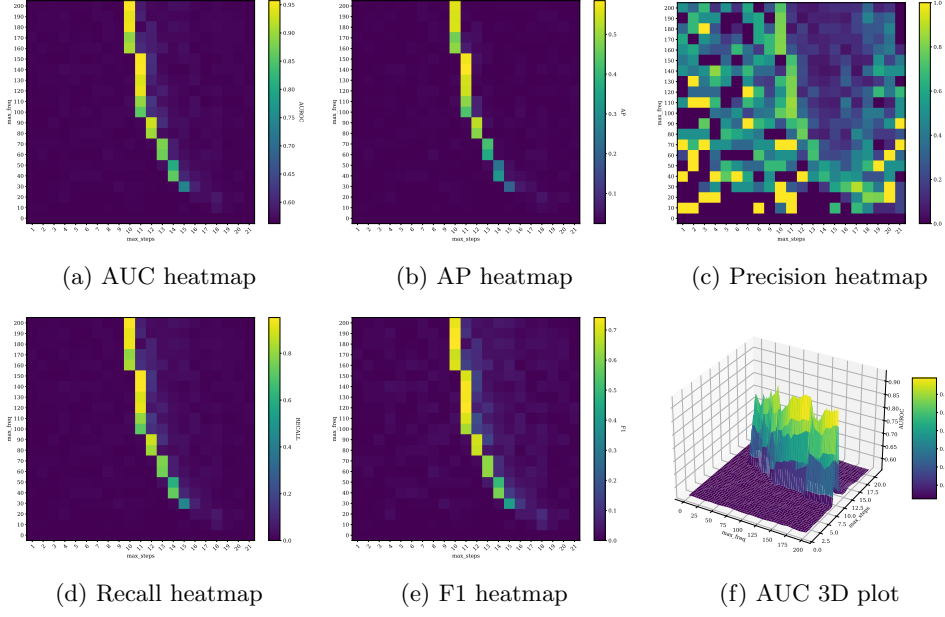


Fig. 12: T_F (y-axis) and $\max_steps$ (x-axis) parameter analysis on the Amazon dataset.

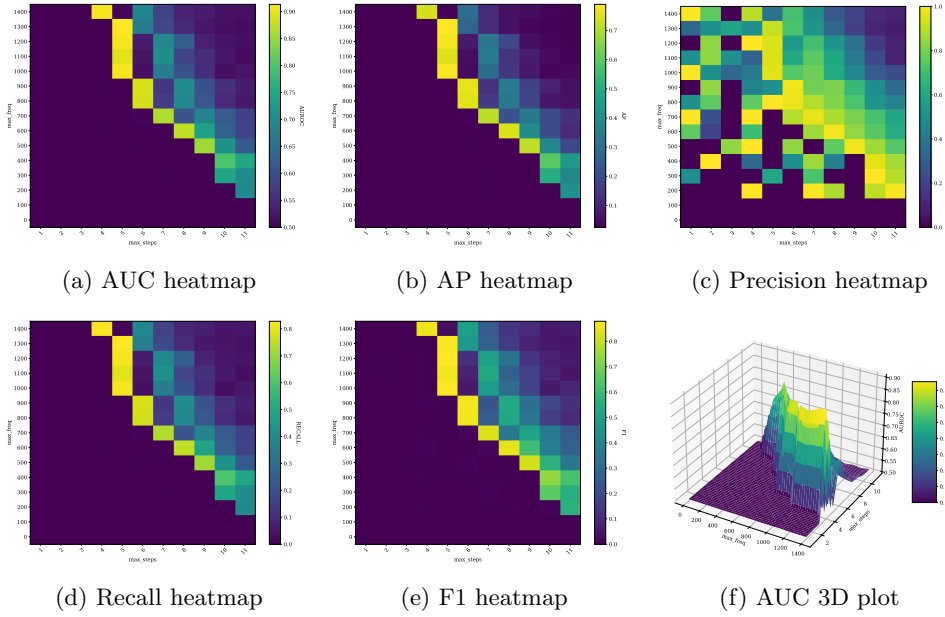


Fig. 13: T_F (y-axis) and $\max_steps$ (x-axis) parameter analysis on the Flickr dataset.

4.4 Robustness to Injection Type and Ratio

The experiments above inject dense cliques at a fixed ratio. To test whether Minomally generalizes beyond this setting, we evaluate it on the CiteSeer citation network with a different, non-clique structural injection and across a range of injection ratios. We use *DICE-structural* injection, which rewires a group’s edges toward distant nodes and connects the group internally, producing a structural anomaly that is not a simple clique. We vary the fraction of injected anomalous nodes from 0.5% to 8.1%, well beyond the single 5% clique setting, keeping the DSAN matcher and the search procedure fixed.

Table 12 reports the results. Minomally detects this non-clique injection reliably across the whole range, with AUC between 91.7% and 97.0% and F1 between 85.2% and 92.0%. Detection quality is consistently high across all ratios, with precision remaining above 80% throughout. At the lowest ratio (0.5%, only 18 anomalous nodes), the method still achieves 84.2% precision and 88.9% recall. As the ratio grows, precision increases further because larger injected groups are easier to separate from normal neighborhoods, reaching 96.7% at 8.1%. This confirms that the approach is not specific to dense cliques at a single ratio: it detects structural anomalies of a different topology across an order of magnitude of injection ratios.

Table 12: Minomally on CiteSeer with DICE-structural (non-clique) injection across injection ratios. Detection is robust from 0.5% to 8.1%.

Injection ratio	AUC	AP	Recall	Precision	F1
0.5%	94.4	79.5	88.9	84.2	86.5
0.8%	92.5	76.9	85.2	85.2	85.2
1.4%	95.4	72.6	91.1	80.4	85.4
2.7%	97.0	86.4	94.4	81.7	87.6
4.1%	91.7	76.7	83.7	91.9	87.6
5.4%	95.8	90.1	91.7	85.5	88.5
8.1%	93.8	85.9	87.8	96.7	92.0

4.5 Application to Real-World Fraud Detection

The experiments above use synthetically injected anomalies because, to our knowledge, no public graph anomaly detection benchmark provides labels that explicitly distinguish structural anomalies from contextual or behavioral ones. To evaluate Minomally in a setting where labels are organic rather than injected, we turn to a real-world fraud detection dataset and ask whether the structural anomalies that Minomally discovers correspond to genuine fraudulent activity.

4.5.1 Dataset and Structural Analysis

We use the Amazon fraud detection graph introduced by [67], which models user reviews on Amazon. The co-purchase (U-P-U) relation connects two users if they review the same product. This graph has 11,944 nodes, 351,216 edges, and 821 labeled fraud nodes (6.9%). Unlike the injected benchmarks, no structural anomaly labels exist. We therefore analyze the fraud subgraph to identify which fraud nodes participate in structurally anomalous formations. Specifically, we extract connected components among fraud nodes and compute each component’s internal edge density.

Seven components with five or more nodes and density above 0.8 emerge, containing 136 fraud nodes in total (16.6% of all fraud). The largest is a 79-node near-clique with density 0.95, and four components are perfect 10-node or 5-node cliques. The remaining 685 fraud nodes are scattered throughout the graph with no structural signature. This analysis confirms that real-world fraud contains a structurally anomalous component that a topology-based method can target.

4.5.2 Detection Results

Minomally runs on the full graph with no knowledge of labels, using the DSAN matcher trained on synthetic graphs. The beam search verifies 6,765 anchor nodes as belonging to infrequent neighborhoods, of which 439 are fraud (53.5% recall against all fraud). The fraud cliques are rare enough to fall within the infrequent band and be discovered by the search. Since this dataset has no structural anomaly annotations, verified patterns include both fraudulent and non-fraudulent rare structures. Minomally returns each verified pattern as an explicit subgraph, so the analyst can inspect what kind of rare structure was discovered and interpret it accordingly.

To characterize the discovered patterns, we define two structural features for every node in the graph. First, the *verified-neighbor count* (vn_count): how many of the node’s graph neighbors are themselves verified anchors. Second, the *verified-neighbor density* (vn_density): among those verified-anchor neighbors, the fraction of all possible edges that actually exist. A node embedded in a fraud clique has many verified neighbors that are all connected to each other, producing $\text{vn_density} \approx 1.0$. A benign node may have a few verified neighbors, but they are scattered and unrelated, producing $\text{vn_density} \approx 0.0$. Dense cliques are a well-known structural signature of coordinated fraud, and filtering the discovered patterns for this topology isolates the fraudulent structures from the rest.

Table 13 quantifies this separation. True-positive anchors (structural fraud nodes that Minomally verified) have a median verified-neighbor density of 1.000 and a median verified-neighbor count of 116, while false-positive anchors have a median density of 0.000 and a median count of 1. The composite score $\log(\text{vn_count} + 1) \times \text{vn_density}$ achieves an AUC of 94.1% against the 136 structural fraud nodes. At the top 500 ranked nodes, 92 of the 136 structural fraud nodes are recovered (67.6% recall at 18.4% precision); at the top 1,000, recovery reaches 113 of 136 (83.1%). Table 14 shows per-clique recovery: the 79-node clique is 97% recovered, the 17-node clique 94%, and two 10-node cliques 100%.

Table 13: Structural profile of true-positive (structural fraud) and false-positive (benign) verified anchors on the Amazon fraud detection graph. The verified-neighbor density separates the two groups by two orders of magnitude.

	Degree	VN count	VN density	Composite
TP anchors (struct. fraud, $n=40$)	418	116	1.000	4.76
FP anchors (benign, $n=2,535$)	8	1	0.000	0.00

Table 14: Per-clique recovery on the Amazon fraud graph. Post-filter composite score (top 1,000 nodes). All large cliques are nearly fully recovered; only the smallest (5-node) cliques fall below the ranking threshold.

Clique	Nodes	Density	Recovered (top-1000)
C_0	79	0.95	77/79 (97%)
C_1	17	0.89	16/17 (94%)
C_2	10	1.00	10/10 (100%)
C_3	10	1.00	10/10 (100%)
C_4	10	1.00	0/10 (0%)
C_5	5	1.00	0/5 (0%)
C_6	5	1.00	0/5 (0%)
Total	136		113/136 (83%)

4.5.3 Implications

Three conclusions follow from this experiment. First, structural anomalies are present in real-world fraud and are not an artifact of synthetic injection. Among the 821 fraud nodes, 16.6% form dense cliques that are topologically distinguishable from the rest of the graph. A method that detects structural anomalies can recover these without any label supervision.

Second, Minomally’s interpretability enables domain-specific filtering of the discovered patterns. Because Minomally returns explicit subgraph patterns rather than opaque scores, the analyst can characterize each pattern structurally and filter for the clique topology that indicates coordinated fraud. This interpretation step achieves AUC 94.1% against the structural fraud nodes. Methods that produce only a score would require additional post-hoc explanation; Minomally provides the structure directly.

Third, the remaining 84% of fraud nodes are behavioral anomalies with no structural signature. Detecting them requires contextual features such as review text, timestamps, or cross-relation patterns. Integrating such features into the order-embedding framework is a natural extension of Minomally and a direction for future work. The experiment demonstrates that structural and contextual anomalies are complementary: a complete fraud detection system should address both, and Minomally provides the structural foundation.

5 Discussion

After presenting the results, we now highlight the key insights from our experiments. Minomally not only outperforms existing methods but also provides a deeper understanding of structural anomaly detection in graphs.

Shifting from Reconstruction to Frequency-Based Detection

Minomally’s performance advantages across datasets validate our fundamental hypothesis that frequency-based anomaly detection captures structural abnormalities more effectively than reconstruction-based approaches. While reconstruction methods must compress graph information into lower-dimensional embeddings before reconstruction,

potentially losing subtle structural differences, Minomally directly measures structure rarity with minimal information loss. Traditional approaches often produce false positives or miss anomalies by directly using reconstruction error magnitude as their core measure of abnormality—assuming difficult-to-reconstruct structures are inherently anomalous. In contrast, Minomally leverages a rich set of embeddings from multiple subgraphs and uses GNN-based comparisons to precisely quantify structure frequency without relying on a single reconstruction outcome. This approach provides a more deterministic and interpretable measure of abnormality compared to the often ambiguous nature of reconstruction errors. The conceptual shift proves particularly valuable on the Amazon dataset, where Minomally achieves an F1 score of 68.5% compared to 21.7% for the next best competitor, demonstrating superior discrimination between truly anomalous and normal structural patterns.

Efficient and Scalable Search Strategy

Minomally’s selective processing of starting nodes (79.5% of Cora nodes, 72.2% of Amazon nodes, and 48.8%–61.2% of Flickr nodes) significantly reduces computation complexity without compromising detection quality. The high precision of these starting nodes (98.6% in Cora, 89.1% in Amazon) shows that we effectively identify regions containing anomalies. Unlike competing methods that suffer from memory limitations due to neighborhood explosion when reconstructing node representations, Minomally remains scalable even on large graphs like Flickr (achieving 91.0%–93.2% AUC). This scalability comes from our frequency-based approach using a sufficiently k embedded random small graphs RG to cover diverse graph patterns, avoiding the memory-intensive operations that cause other methods to fail on larger datasets.

Interpretable Parameter Space

The analysis of T_F and `max_steps` reveals that Minomally’s hyperparameters have clear semantic meaning with predictable effects on performance. The continuous regions of high performance in Figures 11-13 suggest that finding effective configurations is straightforward compared to deep learning approaches. This interpretability facilitates practical deployment and tuning in real-world scenarios. Similarly, the `max_cands` parameter effectively balances exploration thoroughness and computational complexity, as evidenced by strong performance on Amazon (97.62% AUC) despite examining only 20 candidates per step in this highly connected graph.

Integrated Detection and Explanation

Unlike other methods that explain results afterward, Minomally unifies detection and explanation into a coherent process. The frequency curves and anomalous structures in Figure 9 and Tables 8-10 provide transparent rationales for anomaly flags, making results more actionable for domain experts. Figure 10 visualizes how the search process effectively navigates the embedding space to identify anomalous regions, demonstrating that this integrated approach successfully captures meaningful anomalous structures.

925 *Structural Anomalies in Real-World Fraud*

926 The Amazon fraud experiment reveals that structural anomalies are not an artifact
927 of synthetic injection. Among 821 organically labeled fraud nodes, 136 form dense
928 cliques that Minomally discovers without label supervision. Because Minomally returns
929 explicit subgraph patterns, the analyst can characterize each pattern and identify the
930 dense cliques that indicate coordinated fraud. Fraud clique members produce verified-
931 neighbor density of 1.0, while other rare structures produce 0.0. Filtering for this clique
932 topology achieves AUC 94.1% against the structural fraud. Dense cliques are a well-
933 established indicator of coordinated fraud, and Minomally provides a principled way
934 to discover and characterize them. The experiment also delineates the boundary of
935 structural methods: 84% of fraud is behavioral and invisible to topology, motivating
936 the integration of contextual features as future work.

937 These findings collectively demonstrate that Minomally’s frequency-based approach
938 offers a robust foundation for structural anomaly detection that addresses funda-
939 mental limitations in current methods. By directly quantifying node structure rarity,
940 selectively exploring the graph, and providing transparent interpretations, our method
941 achieves superior performance while maintaining computational feasibility for large-
942 scale applications. The clear interpretability of both the method and its results makes
943 it particularly suitable for practical deployment in domains where understanding
944 anomalies is as important as detecting them.

945 **6 Conclusion**

946 We introduced **Minomally**, an unsupervised framework for detecting node structural
947 anomalies in static graphs using subgraph mining and Graph Neural Networks. Our
948 work formalizes structural anomaly detection through subgraph mining with theoretic-
949 al foundations via the anti-monotonicity property, while generalizing structural node
950 anomaly detection across multiple contexts.

951 Minomally applies order embeddings to graph anomaly detection, leveraging GNNs
952 to efficiently encode graph structures. By exploiting the geometrical properties of
953 the Order Embedding Space, our approach significantly reduces verification overhead
954 during anomalous structure search. We developed a computationally efficient greedy
955 search algorithm that enables understandable graph anomaly detection by identifying
956 and interpreting unusual structures.

957 Our comprehensive experiments demonstrate Minomally’s superior performance
958 across all metrics while maintaining scalability on large datasets like Flickr. Unlike
959 reconstruction-based methods, our approach uses frequency as a measure of abnor-
960 mality with interpretable and controllable hyperparameters that can be manually
961 tuned, avoiding memory issues while delivering better performance. [The experiment](#)
962 [on the Amazon fraud detection graph](#) further demonstrates that Minomally generalizes
963 beyond synthetic injection to real-world settings: the search discovers the fraud cliques
964 as rare structures, and the structural interpretation of the returned patterns identifies
965 them as dense cliques indicative of coordinated fraud (AUC 94.1% against struc-
966 turally anomalous fraud). Future work will incorporate node attributes for contextual
967 anomalies, develop more robust node candidate sampling strategies to optimize the

anomalies search for highly connected graphs, and extend the framework to dynamic graphs with temporal order embeddings and graph-level anomaly detection. A further direction is a formal study of the gap between the true subgraph frequency and the OES-based estimate, characterizing the conditions under which the learned order embedding preserves the exact containment-frequency ranking. The fraud experiment highlights that structural and contextual anomalies are complementary: 16.6% of real-world fraud forms dense cliques that Minomaly recovers, while the remaining behavioral fraud requires contextual features. Building a method that integrates both is the natural next step. Advancing structural anomaly detection is fundamental to addressing contextual anomalies due to their intrinsically related nature.

References

- [1] Akoglu, L., Tong, H., Koutra, D.: Graph based anomaly detection and description: a survey. *Data mining and knowledge discovery* **29**, 626–688 (2015)
- [2] Ma, X., Wu, J., Xue, S., Yang, J., Zhou, C., Sheng, Q.Z., Xiong, H., Akoglu, L.: A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* **35**(12), 12012–12038 (2023)
- [3] Ruff, L., Kauffmann, J.R., Vandermeulen, R.A., Montavon, G., Samek, W., Kloft, M., Dietterich, T.G., Müller, K.-R.: A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE* **109**(5), 756–795 (2021)
- [4] Liu, K., Dou, Y., Zhao, Y., Ding, X., Hu, X., Zhang, R., Ding, K., Chen, C., Peng, H., Shu, K., *et al.*: Bond: Benchmarking unsupervised outlier node detection on static attributed graphs. *Advances in Neural Information Processing Systems* **35**, 27021–27035 (2022)
- [5] Bandyopadhyay, S., N, L., Vivek, S.V., Murty, M.N.: Outlier resistant unsupervised deep architectures for attributed network embedding. In: *Proceedings of the 13th International Conference on Web Search and Data Mining*, pp. 25–33 (2020)
- [6] Xu, Z., Huang, X., Zhao, Y., Dong, Y., Li, J.: Contrastive attributed network anomaly detection with data augmentation. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp. 444–457 (2022). Springer
- [7] Yuan, X., Zhou, N., Yu, S., Huang, H., Chen, Z., Xia, F.: Higher-order structure based anomaly detection on attributed networks. In: *2021 IEEE International Conference on Big Data (Big Data)*, pp. 2691–2700 (2021). IEEE
- [8] Angelov, P.P., Soares, E.A., Jiang, R., Arnold, N.I., Atkinson, P.M.: Explainable artificial intelligence: an analytical review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **11**(5), 1424 (2021)
- [9] Bressan, M., Leucci, S., Panconesi, A.: Faster motif counting via succinct color coding and adaptive sampling. *ACM Transactions on Knowledge Discovery from*

- 1005 Data (TKDD) **15**(6), 1–27 (2021)
- 1006 [10] He, Z., Xu, X., Huang, J.Z., Deng, S.: A frequent pattern discovery method
1007 for outlier detection. In: Advances in Web-Age Information Management: 5th
1008 International Conference, WAIM 2004, Dalian, China, July 15-17, 2004 5, pp.
1009 726–732 (2004). Springer
- 1010 [11] Yan, X., Han, J.: gspan: Graph-based substructure pattern mining. In: 2002 IEEE
1011 International Conference on Data Mining, 2002. Proceedings., pp. 721–724 (2002).
1012 IEEE
- 1013 [12] Nassif, A.B., Talib, M.A., Nasir, Q., Dakalbab, F.M.: Machine learning for
1014 anomaly detection: A systematic review. *Ieee Access* **9**, 78658–78700 (2021)
- 1015 [13] Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y.: Adbench: Anomaly detection
1016 benchmark. *Advances in neural information processing systems* **35**, 32142–32159
1017 (2022)
- 1018 [14] Agrawal, S., Agrawal, J.: Survey on anomaly detection using data mining
1019 techniques. *Procedia Computer Science* **60**, 708–713 (2015)
- 1020 [15] Blázquez-García, A., Conde, A., Mori, U., Lozano, J.A.: A review on out-
1021 lier/anomaly detection in time series data. *ACM computing surveys (CSUR)*
1022 **54**(3), 1–33 (2021)
- 1023 [16] Schmidl, S., Wenig, P., Papenbrock, T.: Anomaly detection in time series: a com-
1024 prehensive evaluation. *Proceedings of the VLDB Endowment* **15**(9), 1779–1797
1025 (2022)
- 1026 [17] Benkabou, S.-E., Benabdeslem, K., Kraus, V., Bourhis, K., Canitia, B.: Local
1027 anomaly detection for multivariate time series by temporal dependency based on
1028 poisson model. *IEEE Transactions on Neural Networks and Learning Systems*
1029 **33**(11), 6701–6711 (2021)
- 1030 [18] Benkabou, S.-E., Benabdeslem, K., Canitia, B.: Unsupervised outlier detection
1031 for time series by entropy and dynamic time warping. *Knowledge and Information*
1032 *Systems* **54**(2), 463–486 (2018)
- 1033 [19] Chalapathy, R., Chawla, S.: Deep learning for anomaly detection: A survey. *arXiv*
1034 *preprint arXiv:1901.03407* (2019)
- 1035 [20] Choi, K., Yi, J., Park, C., Yoon, S.: Deep learning for anomaly detection in
1036 time-series data: Review, analysis, and guidelines. *IEEE access* **9**, 120043–120065
1037 (2021)
- 1038 [21] Li, J., Dani, H., Hu, X., Liu, H.: Radar: Residual analysis for anomaly detection
1039 in attributed networks. In: *IJCAI*, vol. 17, pp. 2152–2158 (2017)

- [22] Peng, Z., Luo, M., Li, J., Liu, H., Zheng, Q., *et al.*: Anomalous: A joint modeling approach for anomaly detection on attributed networks. In: IJCAI, vol. 18, pp. 3513–3519 (2018)
- [23] Mahoney, M.W., Drineas, P.: Cur matrix decompositions for improved data analysis. *Proceedings of the National Academy of Sciences* **106**(3), 697–702 (2009)
- [24] Xu, X., Yuruk, N., Feng, Z., Schweiger, T.A.: Scan: a structural clustering algorithm for networks. In: *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 824–833 (2007)
- [25] Zhang, Z., Cui, P., Zhu, W.: Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* **34**(1), 249–270 (2020)
- [26] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y.: A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* **32**(1), 4–24 (2020)
- [27] Wu, S., Sun, F., Zhang, W., Xie, X., Cui, B.: Graph neural networks in recommender systems: a survey. *ACM Computing Surveys* **55**(5), 1–37 (2022)
- [28] Ding, K., Li, J., Bhanushali, R., Liu, H.: Deep anomaly detection on attributed networks. In: *Proceedings of the 2019 SIAM International Conference on Data Mining*, pp. 594–602 (2019). SIAM
- [29] Fan, H., Zhang, F., Li, Z.: Anomalydae: Dual autoencoder for anomaly detection on attributed networks. In: *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5685–5689 (2020). IEEE
- [30] Chen, Z., Liu, B., Wang, M., Dai, P., Lv, J., Bo, L.: Generative adversarial attributed network anomaly detection. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 1989–1992 (2020)
- [31] Hooi, B., Song, H.A., Beutel, A., Shah, N., Shin, K., Faloutsos, C.: Fraudar: Bounding graph fraud in the face of camouflage. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 895–904 (2016)
- [32] Jiang, J., Li, Y., He, B., Hooi, B., Chen, J., Kang, J.K.Z.: Spade: A real-time fraud detection framework on evolving graphs. *Proceedings of the VLDB Endowment* **16**(3), 461–469 (2023)
- [33] Jiang, J., Chen, Y., He, B., Chen, M., Chen, J.: Spade+: A generic real-time fraud detection framework on dynamic graphs. *IEEE Transactions on Knowledge and Data Engineering* **36**(11), 7058–7073 (2024) <https://doi.org/10.1109/TKDE>.

- [34] Kakkad, J., Jannu, J., Sharma, K., Aggarwal, C., Medya, S.: A survey on explainability of graph neural networks. *arXiv preprint arXiv:2306.01958* (2023)
- [35] Wu, B., Bian, Y., Zhang, H., Li, J., Yu, J., Chen, L., Chen, C., Huang, J.: Trustworthy graph learning: Reliability, explainability, and privacy protection. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4838–4839 (2022)
- [36] Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems* **32** (2019)
- [37] Yang, Y., Wang, P., He, X., Zou, D.: Gram: An interpretable approach for graph anomaly detection using gradient attention maps. *Neural Networks* **178**, 106463 (2024)
- [38] Liu, Y., Ding, K., Lu, Q., Li, F., Zhang, L.Y., Pan, S.: Towards self-interpretable graph-level anomaly detection. *Advances in Neural Information Processing Systems* **36**, 8975–8987 (2023)
- [39] Zhou, H., Zhang, Y., He, L., Shen, L., Chen, B.Y.: Interpretable graph convolutional network for alzheimer’s disease diagnosis using multi-modal imaging genetics. In: *2023 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pp. 1004–1007 (2023). IEEE
- [40] Sun, M., Zhao, S., Gilvary, C., Elemento, O., Zhou, J., Wang, F.: Graph convolutional networks for computational drug development and discovery. *Briefings in bioinformatics* **21**(3), 919–935 (2020)
- [41] Shneiderman, B.: Bridging the gap between ethics and practice: guidelines for reliable, safe, and trustworthy human-centered ai systems. *ACM Transactions on Interactive Intelligent Systems (TiiS)* **10**(4), 1–31 (2020)
- [42] Mansouri, D.E.K., Benkabou, S.-E., Meddahi, K., Hadjali, A., Mesmoudi, A., Benabdeslem, K., Chaib, S.: Cosp: co-selection pick for a global explainability of black box machine learning models. *World Wide Web* **26**(6), 3965–3981 (2023)
- [43] Potin, L., Figueiredo, R., Labatut, V., Largeron, C.: Pattern mining for anomaly detection in graphs: Application to fraud in public procurement. In: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 69–87 (2023). Springer
- [44] Nijssen, S., Kok, J.N.: The gaston tool for frequent subgraph mining. *Electronic Notes in Theoretical Computer Science* **127**(1), 77–87 (2005)

- 1111 [45] Guo, Q., Yang, J., Zhang, W., Wang, H., Zhang, Y., Lin, X.: Efficient and accurate
1112 subgraph counting: A bottom-up flow-learning based approach. *Proceedings of*
1113 *the VLDB Endowment* **18**(8), 2695–2708 (2025)
- 1114 [46] Ying, R., Fu, T., Wang, A., You, J., Wang, Y., Leskovec, J.: Representation
1115 learning for frequent subgraph mining. *arXiv preprint arXiv:2402.14367* (2024)
- 1116 [47] Liu, X., Zhang, L., Sun, J., Yang, Y., Yang, H.: D2match: leveraging deep learning
1117 and degeneracy for subgraph matching. In: *International Conference on Machine*
1118 *Learning*, pp. 22454–22472 (2023). PMLR
- 1119 [48] McFee, B., Lanckriet, G.: Partial order embedding with multiple kernels. In: *Pro-*
1120 *ceedings of the 26th Annual International Conference on Machine Learning*, pp.
1121 721–728 (2009)
- 1122 [49] Vendrov, I., Kiros, R., Fidler, S., Urtasun, R.: Order-embeddings of images and
1123 language. *arXiv preprint arXiv:1511.06361* (2015)
- 1124 [50] Cook, S.A.: The complexity of theorem-proving procedures. In: *Logic, Automata,*
1125 *and Computational Complexity: The Works of Stephen A. Cook*, pp. 143–152
1126 (2023)
- 1127 [51] Erdős, P., Rényi, A.: On random graphs. *Publ. Math. Debrecen* **6**, 290–297 (1959)
- 1128 [52] Watts, D.J., Strogatz, S.H.: Collective dynamics of ‘small-world’ networks. *nature*
1129 **393**(6684), 440–442 (1998)
- 1130 [53] Barabási, A.-L., Albert, R.: Emergence of scaling in random networks. *science*
1131 **286**(5439), 509–512 (1999)
- 1132 [54] Holme, P., Kim, B.J.: Growing scale-free networks with tunable clustering.
1133 *Physical review E* **65**(2), 026107 (2002)
- 1134 [55] Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large
1135 graphs. *Advances in neural information processing systems* **30** (2017)
- 1136 [56] Wang, X., Zhang, M.: Glass: Gnn with labeling tricks for subgraph representa-
1137 tion learning. In: *International Conference on Learning Representations (ICLR)*
1138 (2022)
- 1139 [57] Calders, T., Ramon, J., Van Dyck, D.: Anti-monotonic overlap-graph support
1140 measures. In: *2008 Eighth IEEE International Conference on Data Mining*, pp.
1141 73–82 (2008). IEEE
- 1142 [58] Kashtan, N., Itzkovitz, S., Milo, R., Alon, U.: Efficient sampling algorithm for
1143 estimating subgraph concentrations and detecting network motifs. *Bioinformatics*
1144 **20**(11), 1746–1758 (2004)

- 1145 [59] Rex, Ying, Lou, Z., You, J., Wen, C., Canedo, A., Leskovec, J.: Neural subgraph
1146 matching. arXiv preprint arXiv:2007.03092 (2020)
- 1147 [60] Liu, F.T., Ting, K.M., Zhou, Z.-H.: Isolation forest. In: 2008 Eighth Ieee
1148 International Conference on Data Mining, pp. 413–422 (2008). IEEE
- 1149 [61] Liu, K., Dou, Y., Ding, X., Hu, X., Zhang, R., Peng, H., Sun, L., Philip, S.Y.:
1150 Pygod: A python library for graph outlier detection. Journal of Machine Learning
1151 Research **25**(141), 1–9 (2024)
- 1152 [62] Gu, J., Zou, D.: Three revisits to node-level graph anomaly detection: Out-
1153 liers, message passing and hyperbolic neural networks. In: Learning on Graphs
1154 Conference, pp. 14–1 (2024). PMLR
- 1155 [63] Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., Eliassi-Rad, T.:
1156 Collective classification in network data. AI magazine **29**(3), 93–93 (2008)
- 1157 [64] Shchur, O., Mumme, M., Bojchevski, A., Günnemann, S.: Pitfalls of graph neural
1158 network evaluation. arXiv preprint arXiv:1811.05868 (2018)
- 1159 [65] Zeng, H., Zhou, H., Srivastava, A., Kannan, R., Prasanna, V.: Graphsaint: Graph
1160 sampling based inductive learning method. arXiv preprint arXiv:1907.04931
1161 (2019)
- 1162 [66] Kipf, T.N., Welling, M.: Variational graph auto-encoders. arXiv preprint
1163 arXiv:1611.07308 (2016)
- 1164 [67] Dou, Y., Liu, Z., Sun, L., Deng, Y., Peng, H., Yu, P.S.: Enhancing graph neural
1165 network-based fraud detectors against camouflaged fraudsters. In: Proceedings
1166 of the 29th ACM International Conference on Information and Knowledge
1167 Management (CIKM), pp. 315–324 (2020). ACM